

The Verifier Standard (VSTD): A Verification Domain Language and Two-Axis Reference Architecture for Portable Computational Evidence

Foundational Monograph on Bounded Refutable Proofs, Closure Coordinates, and
Superintelligence Oracle Stratification

Tyler Roost

Preprint, version 2.0.0 release candidate — UNRELEASED

Abstract

The Verifier Standard (VSTD) is a verification domain language and reference architecture for portable, bounded, refutable evidence about computational claims. Operating along two orthogonal axes—VSTD-1..5 for single-object mechanics and VSTD-Graph-1..5 for dynamic collection dynamics—VSTD standardizes claim boundaries and portable result semantics across domain verifiers without replacing their native execution engines. The central impetus of the standard is to discover routes toward definably complete representations capable of handling hypercomputation-like oracle classes expected of superintelligences, organizing recursive reflection and verification stages into well-founded transfinite strata without collapsing into unearned assurance, self-observational bias, or semantic ambiguity.

Contents

1	Orientation, Seven Domain Lenses, and First Principles	3
1.1	1. The 30-second intuition: grocery receipts vs. green checkmarks	3
1.2	2. Why “UNKNOWN” is your friend	4
1.3	3. Five-minute Python walkthrough	4
1.3.1	Step 1: Install <code>verifier-standard</code>	4
1.3.2	Step 2: Inspect the library	4
1.3.3	Step 3: Run the built-in defensive demonstration	4
1.4	4. Seven domain lenses: how VSTD fits your field	5
1.4.1	A. For Undergraduates, High Schoolers & Self-Taught Programmers	5
1.4.2	B. For Boolean Satisfiability (SAT) and Satisfiability Modulo Theories (SMT) Solver Enthusiasts	5
1.4.3	C. For Compiler and Systems Engineers	6
1.4.4	D. For AI Safety Researchers and Transfinite Mathematicians	6
1.4.5	E. For Intellidynamics and AI Benchmarkers	6
1.4.6	F. For Deep Mathematics & Theorem Provers (Lean 4, Metamath, and Formal Kernels)	6
1.4.7	G. For Nanochemistry, Molecular Robotics & Mechanosynthesis (Self-Assembly Containment and Anti-Replication Gates)	7
1.5	5. Next steps	7
2	The Two-Axis Profile Taxonomy (LADDER)	8
2.0.1	Read this first: the number is a checklist position, not a strength score	8
2.0.2	Terminology contract	9
2.1	1. The governing idea	11
2.1.1	1.1 Artifact-first causal provenance orientation	11
2.2	2. The object profile axis	13
2.2.1	2.1 The single-declarant boundary	14
2.3	3. The Graph axis	14
2.4	4. Why satisfiability is the spine	16
2.4.1	4.1 Some receipt fields are checkable certificates	16
2.4.2	4.2 Bounded admission uses CNF	16
2.4.3	4.3 Global completeness is outside the observation boundary	16
2.5	5. Certificates for refusals	16
2.5.1	5.1 The internal VSTD-4 rung sequence	17
2.6	6. Composition — closure coordinates do not supply or substitute	17
2.7	7. Numbering	17
2.8	8. Intellectual lineage and adjacent precedents	18
3	Grounded Certification Contracts and Mechanism Boundaries	20
3.1	1. Coordinates and meaning	20
3.2	2. Admission and grounding	20
3.3	3. Evaluation and dependency semantics	21

3.4	4. Portability, bounds and compatibility	21
3.5	5. Object-profile obligation catalogue	22
3.5.1	VSTD-1: Claim Mechanics	22
3.5.2	VSTD-2: Verification Surface	23
3.5.3	VSTD-3: Substrate Accountability	24
3.5.4	VSTD-4: Refutability	26
3.5.5	VSTD-5: Witness Corroboration	28
4	Version 2 Candidate Scope and Release Boundaries	31
4.1	Release name and grounded certification	31
4.2	Recovered scope and lifecycle	32
4.3	Compatibility and migration	33
4.4	Publication gates	34
5	Formal System Architecture and Kernel Isolation	35
5.1	Numbered-profile ownership	35
5.2	Verification complex and profile satisfaction	38
5.3	Operational traversal and recursive Graph materialization	38
5.3.1	Experimental interoperability planning side path	39
5.4	Governing ZIZK architecture and mechanism ownership	40
5.5	Serialized receipt dispatch	41
5.6	Installed specification ownership	42
5.7	Generic-run validation contract	42
5.7.1	Generic-run assessment context	42
5.8	Five-As human traversal	43
5.8.1	Recursive-amplification falsification outcomes	45
5.9	Separation and Graph boundaries	46
5.9.1	Recursive current-state audit	47
6	Claims, Limits, Boundaries, and Refutation Mechanics	49
6.1	Reading every VSTD claim	49
6.2	Skeptical review summary	49
6.3	Claim translation table	52
6.4	Competition and scored-evaluation claims	57
6.5	VSTD-4 grounded-decision claim translations	57
6.6	VSTD-3 accelerator claim translations	58
6.6.1	What the current adapters can say	61
6.7	Implemented mechanism versus specification requirement	61
6.8	Safe claim template	61
6.9	Prohibited shortcuts	62
7	Conceptual Foundations, Precedents, and Adjacent Standards	63
7.1	Orientation glossary	63
7.2	Primary reference map	68
7.3	How to cite these precedents	70
8	Platform Interoperability and Hardware Envelopes	71
8.1	Evidence coordinates and freshness	71
8.2	Recorded candidate observation	71
8.3	Evidence classes	72
8.4	Evidence-surface summary	73
8.5	Machine-checked reference-component contract intent	74
8.6	Reading a platform-comparison result	90
8.7	Release maintenance rule	90

Chapter 1

Orientation, Seven Domain Lenses, and First Principles

Acronyms used below: application programming interface (API); artificial intelligence (AI); Boolean satisfiability problem (SAT); central processing unit (CPU); command-line interface (CLI); conjunctive normal form (CNF); deletion resolution asymmetric tautology (DRAT); directed acyclic graph (DAG); grounded decision certificate (GDC); identifier (ID); intermediate representation (IR); JavaScript Object Notation (JSON); linear resolution asymmetric tautology (LRAT); machine learning (ML); nondeterministic polynomial time (NP); operating system (OS); reduced instruction set computer (RISC); satisfiability modulo theories (SMT); Secure Hash Algorithm 256-bit (SHA-256); unsatisfiable (UNSAT); Verifier Standard (VSTD).

Welcome to Verifier Standard (VSTD). If you are new to formal verification, proof systems, or computational reproducibility, this guide is designed for you. Whether you are a high school student, an undergraduate taking your first data structures course, a compiler developer, an AI safety researcher, or a SAT solver expert, this guide introduces the core concepts and gets you running code in five minutes.

1.1 1. The 30-second intuition: grocery receipts vs. green checkmarks

When you write software or run experiments today, tools usually give you a single bit of feedback: a green checkmark (**PASS**) or a red cross (**FAIL**).

Imagine going to the grocery store, handing over your card, and the cashier says: *“Payment complete, trust me.”* You would ask: - *Which items were scanned?* - *What was the unit price of each item?* - *What store, date, and cash register processed this?* - *If there is an overcharge on line 4, how can I dispute that exact item?*

You expect an **itemized receipt** that binds the transaction to the exact items, register, and timestamp.

In scientific computing, machine learning, and software engineering, a green checkmark does not tell you: 1. **What exact file or dataset was checked?** (Did someone swap the weights or modify the dataset after the run?) 2. **What tool did the checking, and what were its bounds?** (Did it run out of time and exit 0 by mistake?) 3. **Can another person reproduce the exact same bytes on their machine?** (Or does it only work on the author’s laptop?) 4. **How can someone prove the result wrong?** (What is the refutation recipe?)

VSTD is the itemized receipt for computational claims.

It wraps any computation with five cryptographic guarantees:

```
claim + exact evidence + named mechanism + bounds + refutation recipe
      |
      v
    PASS / FAIL / UNKNOWN
```

1.2 2. Why “UNKNOWN” is your friend

In standard tools, everything is forced into a binary: either it passed or it failed. If a test times out, a script crashes, or a theorem prover runs out of memory, systems often silently fail or crash.

In honest science, if your thermometer only measures up to 100°C and your reaction reaches 150°C, you do not write down “*Reaction passed.*” You write down: **inconclusive / out of bounds**.

VSTD elevates UNKNOWN to a first-class, honorable result: - **PASS**: The specific tool checked the specific claim over the specific files within stated resource bounds, and confirmed it. - **FAIL**: The tool found a counterexample or proven violation. - **UNKNOWN**: The tool ran out of time, memory, or evidence before reaching a conclusion. **VSTD refuses to guess**.

1.3 3. Five-minute Python walkthrough

You do not need complicated tools or heavy frameworks to use VSTD. The reference library has **zero required third-party dependencies**—it runs on standard Python 3.10+.

1.3.1 Step 1: Install verifier-standard

```
python -m pip install "verifier-standard==1.5.0"
```

1.3.2 Step 2: Inspect the library

Open a Python interactive shell:

```
import verifier

print(verifier.__version__)
## '1.5.0'

print(verifier.__standard__)
## 'VSTD-5'
```

1.3.3 Step 3: Run the built-in defensive demonstration

To see VSTD catch real-world defects without modifying any files on your computer, run the canonical cross-platform command:

```
vstd demo
```

You will see four defensive scenarios: 1. **Valid-looking proof, wrong artifact (REJECTED)**: An attacker provides a real mathematical proof, but binds it to the wrong data file. VSTD computes the SHA-256 hash and catches the mismatch immediately. 2. **Bound exhausted without a false answer (ACCEPTED/UNKNOWN)**: A calculation hits its resource limit. Instead of faking a pass, VSTD honestly outputs UNKNOWN. 3. **Inflated verification-cost claim (REJECTED)**: A script claims a verification step cost \$1,000,000 of compute when it

only took milliseconds. The receipt refuses the ungrounded cost claim. 4. **Revoked ancestor behind valid descendants (GRAPH-CANDIDATE-0)**: A computation looks valid, but one of its upstream dependencies was revoked. VSTD traces the lineage graph and flags the compromised ancestor.

1.4 4. Seven domain lenses: how VSTD fits your field

VSTD is domain-agnostic: it does not replace your native tools; it packages and standardizes their evidence boundaries. Here is how people in different fields use it, with concrete links to tutorials and runnable examples:

1.4.1 A. For Undergraduates, High Schoolers & Self-Taught Programmers

Think of VSTD as a **tamper-proof digital lab notebook**. When submitting a class project, physics simulation, or machine learning experiment: - You hash your dataset, source code, and output graphs. - You record the exact Python version, command arguments, and operating system. - Anyone grading or evaluating your work can verify with one command that your results were generated by that exact code on that exact data.

1.4.1.1 How can a grader possibly verify your work with one command?

If you have ever handed in homework, you might wonder: *How can a teacher or automated grading bot actually know my code really produced this graph without reading every single line or trusting my word?*

Here is the exact three-step mechanic: 1. **Digital Fingerprints (SHA-256 Hashes)**: VSTD computes cryptographic fingerprints of your code (`script.py`) and inputs (`data.csv`). If anyone edits a single character or changes one number in the comma-separated values (CSV), the fingerprint changes completely into a different, unpredictable hex string. 2. **The Receipt Binding**: When you run `vstd run manifest.json --output my_receipt`, VSTD executes your script, records the exact command, exit code, and runtime platform details (e.g. Python version and OS architecture, without recording private host environment variables or credentials), and takes fingerprints of the output files (e.g. `plot.png` has digest `e3b0c442...`). 3. **The Grader's Single Command (`vstd reproduce`)**: The grader downloads your receipt and runs: `bash vstd reproduce my_receipt/` Under the hood, `vstd reproduce`: - Checks that the inputs match the fingerprints in your receipt. - Creates a clean, temporary execution folder and re-runs `python script.py data.csv`. - Fingerprints the newly generated plot file and compares: *Does the fresh digest match e3b0c442... byte-for-byte?* - If they match, the grader sees: `[REPRODUCE OK] 100% byte-for-byte identical output`. If a student edited the graph in Photoshop or faked the numbers, the newly computed bytes will not match, and VSTD outputs `REJECTED: output digest mismatch`.

- **Tutorial & Runnable Specimen**: Follow the full step-by-step walkthrough in **Your First Receipt** and inspect the runnable specimen in `examples/generic_run/`.

1.4.2 B. For Boolean Satisfiability (SAT) and Satisfiability Modulo Theories (SMT) Solver Enthusiasts

Modern SAT and SMT solvers decide whether a propositional logic formula is satisfiable (SAT) or unsatisfiable (UNSAT). But solver implementations are complex and can have soundness bugs: - For SAT, the solver produces a truth assignment that is trivially checkable in linear time. - For UNSAT, high-assurance solvers emit clausal proof certificates, such as deletion resolution asymmetric tautology (DRAT) or linear resolution asymmetric tautology (LRAT) format traces. - VSTD wraps the conjunctive normal form (CNF) formula digest, certificate file digest, certificate checker binary, and CPU runtime bounds into a standard JSON receipt. Third parties can verify the refutation certificate without repeating the NP-hard search. - **Tutorial & Runnable Specimen**: Read about clausal certificates in Concepts and Precedents: LRAT and DRAT and inspect the executable certificate checker test in `tests/test_gdc_certificate.py`.

1.4.3 C. For Compiler and Systems Engineers

Compilers optimize programs across multiple intermediate representations (IR). A compiler bug can silently miscompile code into incorrect machine instructions: - Translation validation proves that an IR transformation pass preserves program semantics. - VSTD turns optimization passes into verifiable contracts: it binds input IR hash, output IR hash, translation validator tool, target CPU architecture, and OS environment constraints. - Artifact control freezes intermediate artifacts into sealed, immutable bundles signed with cryptographic keys. If a thawed artifact or downstream dependency is modified by even a single byte, verification fails closed (THAWED_DIRTY). - **Tutorial & Runnable Specimen:** Walk through **Seal an Artifact and Detect a Change** and **Publish a Silo Package**.

1.4.4 D. For AI Safety Researchers and Transfinite Mathematicians

When analyzing self-referential systems, recursive agent loops, ordinal bounds, or non-well-founded belief graphs: - Informal safety arguments frequently suffer from circular reasoning, ungrounded induction steps, or silent domain upgrades. - VSTD enforces strict **stratified profiles (Profiles 1 to 5)**: a higher profile cannot supply or backfill missing evidence for a lower profile, and self-observation is never promoted to independent verification. - Graph profiles track directed acyclic graphs (DAGs) and detect when an upstream dependency in an agent’s reasoning chain has been revoked or invalidated. - **Tutorial & Runnable Specimen:** Study the profile hierarchy in the **Normative Ladder** and run the four adversarial specimens in `examples/flagship_demo/`.

1.4.5 E. For Intellidynamics and AI Benchmarkers

Automated agent evaluations and language model leaderboards are plagued by prompt leaking, test-set contamination, and stochastic output variance: - VSTD turns a benchmark score from an unproven assertion (“Model X scored 94%”) into an auditable, refutable evidence bundle. - It records exact prompt digests, model weights identifiers, temperature and random seed coordinates, frozen tool execution traces, and adversarial replay instructions. - Anyone can re-run the exact verification recipe to confirm or falsify the benchmark score. - **Tutorial & Runnable Specimen:** Walk through Publishing a Benchmark Number Somebody Else Can Check using `examples/generic_run/manifest.json`.

1.4.6 F. For Deep Mathematics & Theorem Provers (Lean 4, Metamath, and Formal Kernels)

When working in advanced mathematical foundations, interactive theorem provers (such as Lean 4, Coq, Isabelle, or Metamath) and foundational algebraic kernels produce mechanically checked proofs: - In formal mathematical ecosystems—such as **hypermath** (primitive foundational algebraic kernel, quadrilateral filtration, Lean 4 bridge), **ordinatics** (ordinal arithmetic, Veblen hierarchies, transfinite stage semantics), **grounded-hyperset-theory** (Aczel’s Anti-Foundation Axiom (AFA), accessible pointed graphs (APGs), non-well-founded sets), and **grounded-hypercalculi** (Oracle, Language, Meta, Hyper, Ordinal, and Real Calculi, stratified reflection)—mathematical claims span multiple formal representations. - A Lean 4 proof verifies a proposition relative to Lean’s environment and axioms. But how do you verify cross-system mathematics—for example, connecting a Lean 4 theorem to a Python algebraic kernel, a SAT solver certificate, or a Metamath proof step without trusting a single monolithic system? - VSTD provides the **meta-verification envelope**: 1. **Axiom Enumeration:** It binds the exact explicit axiom set (refusing ungrounded axioms or **sorry** escapes). 2. **Environment & Kernel Binding:** It hashes the exact prover kernel binary, source files, and dependencies. 3. **Heterogeneous Composition:** It allows an ordinal bound in **ordinatics**, a hyperset graph in **grounded-hyperset-theory**, and an equational derivation in **hypermath** to link together into a unified, refutable proof graph. - **Foundations, Tutorial & Runnable Specimen:** Review **Mathematical Foundations Guide**, **Python API Guide: Grounded Decision Certificates**, and the **Normative VSTD-4 Grounded Certificate Specification**.

1.4.7 G. For Nanochemistry, Molecular Robotics & Mechanosynthesis (Self-Assembly Containment and Anti-Replication Gates)

In molecular dynamics simulations, chemical reaction networks (CRNs), and autonomous molecular robotics (such as deoxyribonucleic acid (DNA) origami walkers or mechanosynthetic tooltips), molecular self-assembly poses an urgent containment challenge: preventing runaway autocatalytic self-replication.

A molecular design tool or simulation might assert: “*This molecular machine only constructs the target crystal structure.*” But in reality: - Did the simulation explore parasitic reaction pathways where the machine synthesizes copies of its own catalytic core? - What thermodynamic temperature (T), pressure (P), and solvent concentration bounds were verified? - What happens if the design is sent to an automated robotic chemical synthesizer?

VSTD provides the computational containment seal: 1. **Coordinate and Stoichiometric Binding:** VSTD binds the exact atomic coordinate files, force-field potential parameter files, stoichiometric reaction matrices, and simulation integration steps into an immutable receipt. 2. **Refutation Route for Containment Claims:** An assertion of “*Non-self-replicating under standard physiological conditions*” is paired with an explicit refutation recipe: any researcher or adversary who demonstrates a reachable catalytic cycle (a series of reaction steps with activation energies below threshold $E_a < E_{\text{barrier}}$) leading to exponential autocatalysis overturns the claim. 3. **Synthesis Interlock:** Physical robotic nanofabrication devices can require an Ed25519-signed VSTD receipt proving that containment bounds were verified before releasing chemical feedstocks or beginning automated assembly.

- **Tutorial & Runnable Specimen:** Walk through **Seal an Artifact and Detect a Change** and **Publishing a Benchmark Number Somebody Else Can Check**.
-

1.5 5. Next steps

Now that you understand the mental model: - **Run your first generic computation** — step-by-step tutorial capturing a word-frequency script into a receipt. - **Explore the Python API** — generate and inspect receipts directly from your Python code. - **Read the Normative Ladder** — the formal specification of Profiles 1 through 5. - **Inspect the Architecture Map** — internal module layout and design principles.

Chapter 2

The Two-Axis Profile Taxonomy (LADDER)

Acronyms: application programming interface (API); conjunctive normal form (CNF); Certificate Transparency (CT); deletion resolution asymmetric tautology (DRAT); grounded decision certificate (GDC); JavaScript Object Notation (JSON); National Institute of Standards and Technology (NIST); nondeterministic polynomial time (NP); proof-carrying code (PCC); World Wide Web Consortium provenance vocabulary (PROV); PROV data model (PROV-DM); Protect the Software (PS); Request for Comments (RFC); reverse unit propagation (RUP); Boolean satisfiability problem (SAT); Supply-chain Levels for Software Artifacts (SLSA); satisfiability modulo theories (SMT); SMT library standard (SMT-LIB); Secure Software Development Framework (SSDF); The Update Framework (TUF); unsatisfiable (UNSAT); World Wide Web Consortium (W3C).

Status: project specification (normative for numbering and composition) **Editor:** TimeLordRaps **License:** Apache-2.0

Normative language: The uppercase key words in this series are interpreted as described by [RFC 2119](#) and [RFC 8174](#) only when they appear in all capitals; lowercase uses are ordinary prose.

Reader context: [Concept guide and intellectual precedents](#)

VSTD records separate answers to separate verification questions about an identified claim, its evidence, the mechanism that checked it, and the bounds of that check. It does not collapse those answers into one universal “verified” label or confidence score. The questions, their evidence-bearing relations, and cumulative profiles over them form the VSTD **verification complex**.

2.0.1 Read this first: the number is a checklist position, not a strength score

A numbered profile is a cumulative checklist on one axis. VSTD-3, for example, means that the required questions for VSTD-1, VSTD-2, and VSTD-3 are each established by their own applicable evidence. It does **not** mean “assurance strength 3,” software version 3, or evidence that is three times stronger than VSTD-1.

Here, **established** means that a named mechanism checked the exact required proposition against bound evidence under declared limits. A field, document, or actor merely saying that the proposition passed does not establish it.

Consider one verification object with these separately recorded results:

Object-axis question: VSTD-1 Claim Mechanics

Current result: established

Object-axis question: VSTD-2 Verification Surface

Current result: UNKNOWN

Object-axis question: VSTD-3 Substrate Accountability

Current result: established

Its **object profile depth** is 1. The cumulative checklist cannot skip the missing VSTD-2 result. The VSTD-3 coordinate evidence remains recorded and useful, but it does not fill the VSTD-2 gap or make profile 3 satisfied.

The Graph axis applies a different cumulative checklist to a collection of artifacts and their recorded relations. VSTD-3 and VSTD-Graph-3 therefore ask different questions; the shared number does not make them equivalent.

In one sentence: a **closure coordinate** is one verification question, a **numbered profile** is a cumulative checklist of those questions on one axis, and **profile depth** is the largest uninterrupted prefix of that checklist that is established.

2.0.2 Terminology contract

The rest of the Standard uses the following terms precisely:

Term: Closure coordinate

Plain meaning: One named verification question and its failure class, such as Claim Mechanics or Refutability.

Important boundary: Closure is scoped to that question. VSTD-2 surface closure, Graph provenance closure, refutability closure, and artifact-seal structural closure are different results.

Term: Numbered profile

Plain meaning: A cumulative checklist selected by VSTD-N or VSTD-Graph-N. Profile N requires its named coordinate and every earlier coordinate on the same axis.

Important boundary: A profile number is not a software revision, spatial layer, confidence score, or substitute for the underlying results.

Term: Profile axis

Plain meaning: One ordered family of cumulative checklists. VSTD has an object axis and a Graph axis.

Important boundary: Equal numbers on different axes do not identify equivalent or interchangeable results.

Term: Object profile depth

Plain meaning: For one verification object, start at VSTD-1 and count upward only while every required coordinate remains established. The last uninterrupted number is its depth.

Important boundary: Depth is a compact summary of separately established results, not a new verdict, evidence-strength rating, or permission to ignore a later established coordinate after an earlier gap.

Term: Candidate Graph profile

Plain meaning: The greatest Graph checklist position satisfied by the current caller-supplied ratings.

Important boundary: The current calculation is `NOT_ESTABLISHED` because those ratings are not evidence-bound. It is not a verified Graph profile.

Term: Evidence-bound Graph profile

Plain meaning: The greatest Graph checklist position obtained after rerunning exact member, ancestor, and edge rating mechanisms from content-addressed evidence.

Important boundary: It is established only under the named mechanisms, trust roots, evidence, bounds, lifecycle view, and conflict state.

Term: Grounded certification obligation

Plain meaning: `X.M` selects one separately evidence-bound obligation within object numbered profile `X`.

Important boundary: Profile-specific sequences and cumulative prerequisites are defined in `GROUNDED_CERTIFICATION.md`; the suffix is not a software revision or confidence score.

Term: VSTD-4 rung

Plain meaning: One of the fourteen ordered refutability obligations 4.1 through 4.14.

Important boundary: “Rung” names only this internal sequence, never a top-level VSTD profile.

Term: Verification order

Plain meaning: One adjacent meta-verification order in the VSTD-2 geometry model.

Important boundary: The compatibility names `VerificationLayer` and `verification_layers` do not denote numbered VSTD profiles.

Term: Level

Plain meaning: A retained word in an explicitly named external taxonomy or compatibility identifier, including `ReproducibilityLevel`, `AvailabilityLevel`, `graph_level`, and serialized Graph `level` fields.

Important boundary: In Graph compatibility identifiers, the value is the candidate Graph profile number; “level” is not the governing name for a VSTD profile.

Term: Layer

Plain meaning: An implementation, protocol, or physical stack whose parts are ordered by containment.

Important boundary: It does not name `VSTD-N` or `VSTD-Graph-N`; historical paths such as `verifier.layer4` remain compatibility identifiers only.

Term: Tier

Plain meaning: A declared checker-cost class in `VSTD4-GDC-1`.

Important boundary: It is not a VSTD profile, evidence-strength rating, or actor rating.

“Profile” must also be qualified when confusion is possible: **numbered profile**, **receipt profile**, **application profile**, or **geometry profile**. Likewise, “depth” must be qualified as object profile depth, VSTD-4 normative or candidate depth, or lineage topological depth. The retained `LADDER.md` filename is a stable document path, not the governing topology; this document defines a verification complex. Current public serialized identifiers, fields, class names, functions, and module paths retain their exact compatibility spelling; adjacent prose supplies the precise meaning. Profiles are therefore **requirement-set coordinates**, not spatial layers. A profile is satisfied only when a named mechanism has bound evidence for every required fact; Boolean SAT over caller-supplied assertions establishes only a candidate formula result.

2.1 1. The governing idea

Each closure coordinate names a distinct verification question and failure class. Profile ordering is a composition rule, not logical entailment between coordinates.

The nearest familiar security analogy is [defense in depth](#), but the analogy is limited: VSTD closure coordinates are separately evidenced questions, not interchangeable controls whose mere quantity establishes assurance. Decomposing assurance into named components also has precedent in the Common Criteria, while VSTD deliberately uses different coordinates, evidence rules, and conformance semantics.

Evidence for one closure coordinate never supplies evidence for another. In particular, Refutability evidence does not supply, imply, upgrade, or repair Substrate Accountability, Verification Surface, or Claim Mechanics evidence. A reported object profile depth of `N` is only shorthand for the separately checked results required by profiles 1 through `N`.

Reflection and [metalanguage](#) are useful design analogies for asking what a given verification surface leaves unexamined. VSTD does not claim that Tarski’s [undefinability theorem](#) proves these profile coordinates, that adjacent profiles form formal metalanguages, or that an earlier-profile implementation is logically incapable of describing another coordinate’s failure. The normative requirement is narrower: an implementation **MUST NOT** treat success on one question as evidence for a different question.

2.1.1 1.1 Artifact-first causal provenance orientation

VSTD evaluates bounded propositions about computational processes represented by identified software, executions, evidence, and resulting artifacts. It does not evaluate whether an actor is good, bad, reputable, or worthy of trust. Standing alone, an actor’s identity, popularity, repetition, or reputation **MUST NOT** strengthen an artifact-bound result. A named mechanism **MAY** establish an exact attribution, authorization, or separation proposition by checking the required identity evidence; that result remains an adjacent proposition and **MUST NOT** promote an unrelated computational claim.

Zero identity means zero identity-derived verdict weight, not anonymity or absence of identifiers. **Zero knowledge** means zero unevidenced knowledge is presumed: without a mechanism-earned result for the exact proposition, its state remains `UNKNOWN`. This architectural zero-knowledge rule **MAY** be enclosed by cryptographic zero knowledge when a witness must remain confidential. That enclosure **MUST** bind the exact software or program coordinate, predicate, public commitments, output, proof parameters, and verification mechanism while revealing no more witness information than its declared proof statement. Cryptographic zero knowledge **MUST** be claimed only when a named proof system establishes that property under explicit assumptions; a digest or undisclosed input alone is not such a proof. The resulting support is bearer- and artifact-bound, never prover-identity-bound. **Actor** and **artifact** remain contextual roles, not permanent entity classes: software can be an artifact when created, versioned, or evaluated and an actor when it executes a transformation.

The capitalized terms **TRUST**, **RUST**, and **ROT** are formal VSTD semantic names, not acronyms, numbered-profile receipt verdicts, actor ratings, scalar scores, or references to the Rust programming language. They serialize as typed event kinds only in the non-receipt **verifier-graph-assurance-1** mechanism log. The same bound development graph and its time-indexed lifecycle carry three distinct relations:

development: ancestor artifact --TRUST through a checked transformation--> descendant
lifecycle: recorded TRUST --ROT under typed current-state evidence--> reassessment
diagnosis: descendant deviation --RUST memetic causal backtrace--> ancestor candidates

Memetic propagation is the transmission of claim and evidence state through recorded developmental provenance. The genetic or viral language names this inheritance mechanic: TRUST moves forward into descendant claim space; RUST moves backward toward recorded ancestor states; ROT changes the current admissibility of previously recorded support. It does not claim biological transmission or make identity and reputation sources of assurance.

TRUST is positive support earned when a named mechanism checks an exact artifact-bound process obligation under declared evidence, specification, bounds, and trust roots. It moves parent-to-child only across a declared creation or dependency edge whose relevant transformation obligations pass. Applicable support composes by intersection and is capped by the weakest required parent or edge; it is never added, averaged, voted, or converted into actor standing. Every child **MUST** still discharge its new predicates, transformations, boundaries, and evidence obligations. A declared trust root is an explicit dependency and stopping boundary, not actor TRUST.

The reference event mechanism realizes that rule edge by edge. Each TRUST event binds the historical Graph digest, one exact transformation, its complete input artifact set, one output artifact, and the exact prerequisite TRUST event for every derived input. A descendant event is current only while every recursively required event, input, output, and transformation remains admissible and free of an admissibility-blocking conflict. A status-conflict resolution projects its selected state into the current view: **VALID** or **COMPLETED** may restore the affected route, while **REVOKED**, **FAILED**, or another inadmissible state cannot. Resolving an arbitrary predicate selects a retained value but does not establish its admissibility effect, so the route remains blocked. The current reference runtime implements no general non-status admissibility-effect mechanism. Alternate or duplicate paths remain distinct recorded routes; their count supplies no added strength or witness independence.

ROT is typed, time-indexed degradation of the current admissibility of recorded TRUST. It requires exact lifecycle or dependency evidence, such as expiry under a declared freshness bound, **STALE**, **CHALLENGED**, **REVOKED**, **SUPERSEDED**, or an invalidated required coordinate. Wall-clock passage, age, or popularity alone **MUST NOT** create ROT. ROT **MUST NOT** rewrite an immutable historical receipt or imply that its historical result was false. It may require reassessment of dependent descendants, but any resulting status change still requires its named policy or mechanism.

RUST is the inverse-TRUST diagnostic mechanic: a typed trace created by an observed descendant deviation from a declared expectation. It moves child-to-parent only through historically recorded contributing creation, input, or transformation paths. Current revocation, challenge, staleness, or conflict can remove a route from current TRUST without erasing it from historical diagnostic ancestry. The inverse is directional and diagnostic, not arithmetic: TRUST and RUST never cancel. Distinct comparable backtraces may concentrate on a shared ancestor and prioritize it for falsification or diagnostic examination. Transferred RUST establishes ancestral reachability, not current admissibility, direct observation, falsehood, or causal responsibility; localization requires additional intervention, ablation, independently bound execution evidence, or an equivalent declared mechanism.

Reference causal localization **MUST** select one exact passing RUST event, bind that event's digest and the exact descendant-deviation proposition digest, confirm the selected artifact is among that event's recorded ancestors, and preserve those coordinates through replay.

BLAME and **GUILT** are bounded artifact-relative diagnostic results, not opposite directions on the Graph. **BLAME** requires a named mechanism to establish that an exact artifact bears responsibility for or materially contributed to an exact localized deviation. **GUILT** requires three separately bound passing components whose coordinates agree: (1) responsibility or material contribution by exact artifact A for exact localized deviation D; (2) applicability to A of exact obligation O under its declared scope, assumptions, exclusions, roots, and bounds; and (3) violation by A of that same O relative to that same D and applicable scope. The final **GUILT** proposition **MUST** bind the exact obligation coordinate, causal-localization event digest, and

all three component digests. A single compound mechanism MAY check the components in one invocation only when it emits three separately bound evaluations; one opaque combined result or nonempty obligation string is insufficient. An existing passing BLAME event can supply the responsibility component only when its exact event digest and all coordinates match. Neither term concerns actor morality, character, identity, reputation, automatic legal liability, or social scoring. Exoneration, innocence, obligation satisfaction, absence of hidden contributors, or not-guilty conclusions require their own exact propositions and mechanisms; a missing component remains UNKNOWN, not evidence of the opposite. The localization event transitively binds the selected RUST event and exact deviation, so neither result can float across two deviations on the same descendant.

The word *causal* is required here for recorded developmental and provenance causality: the graph states which artifacts and transformations produced later claim architecture. Propagation across those causal-provenance edges does not by itself establish intervention-level physical causality, causal localization, responsibility, or guilt.

TRUST, ROT, and RUST MUST remain separate. They do not cancel, form one scalar score, or flow in the opposite direction as inherited truth, decay, or guilt. UNKNOWN and CONFLICTED support or lineage MUST remain visible and MUST NOT become a clean signal. `verifier-graph-assurance-1` now serializes an additive, hash-chained reference event log with the complete historical Graph, exact proposition bindings, and embedded evidence bytes. `AssuranceLedger` implements mechanism-earned forward TRUST edge by edge, typed ROT, challenge-ledger status projection, reverse RUST reachability, unique-descendant structural concentration, additive conflict declaration and resolution, explicit causal localization, separately bound responsibility/applicability/violation components, and component-composed artifact-relative GUILT. Duplicate paths and repeated records remain set-valued and earn no strength. `recheck_assurance_log` reconstructs the historical Graph, rehashes the embedded evidence, reruns every exact component mechanism—including one-invocation compound groups as such—reproduces the event hash chain, and compares the derived current view. A deployment still supplies the proposition-specific mechanisms: the event format and dispatcher do not establish real-world obligation applicability, legal culpability, a universal support-transfer algebra, or causality from topology.

Artifact freezing and sealing are bounded mechanisms under this orientation, specified separately in `ARTIFACT_CONTROL.md`. A verified freeze preserves and recomputes exact bytes; a verified seal earns structural closure for those bytes. It does not earn semantic correctness, prevent ROT, localize RUST, create actor TRUST, or supply any numbered profile. A sealed realm or temporal descriptor remains a bound input until its own mapping, continuity, or transition verifier checks the exact proposition.

2.2 2. The object profile axis

VSTD proper governs the verification of **one object**. Call this verification *mechanics*.

Numbered profile: 1

Required closure coordinate: Claim mechanics

Closes: A malformed or tampered statement

Does not establish: Whether the claim applies where it is being applied

Numbered profile: 2

Required closure coordinate: Verification surface

Closes: A verdict leaking beyond the coordinate actually verified

Does not establish: Whether the evidence behind it is real

Numbered profile: 3

Required closure coordinate: Substrate accountability

Closes: A lying or unaccountable evidence source

Does not establish: Whether anyone but the declarant could re-derive it

Numbered profile: 4

Required closure coordinate: Refutability

Closes: A claim unfalsifiable in principle by any outside party

Does not establish: Whether the parties who could check are independent

Numbered profile: 5

Required closure coordinate: Witness corroboration

Closes: Pseudo-independence — witnesses sharing the declarant’s trust root

Does not establish: —

2.2.1 2.1 The single-declarant boundary

A single declarant can in principle produce the evidence required by profiles 1 through 4. No second party is required merely to create those bounded inputs and mechanisms.

Profile 5 is not. It requires another party to exist, to act, and to be independent.

VSTD-1 records the claim-mechanics status of actor independence but cannot infer it from two runs or matching artifacts. VSTD-5 requires the corroborating witness procedure that uses such separately evidenced actor participation; recording a field is not witnessing.

That transition between profiles 4 and 5 is the most important object-axis boundary. Refutability asks *is a bounded outside check possible?* Witness Corroboration asks *was one performed, and are the required separation seams evidence-bound?* The first is a property of the claim surface. The second requires additional evidence about an execution.

An implementation **MUST NOT** report a profile-5 property on the basis of Refutability evidence. Preparing to be checked is not being checked.

2.3 3. The Graph axis

VSTD-Graph governs the verification of a **collection** of objects. Call this verification *dynamics*.

The two axes are parallel but coupled: a collection’s dynamics are constrained by its members’ mechanics, and by the [provenance](#) edges between them. The implemented N-ary representation is a [hypergraph](#).

Numbered profile: Graph-1

Required closure coordinate: Recorded lineage

Collection proposition: members and transformations are represented

Numbered profile: Graph-2

Required closure coordinate: Bounded collection surface

Collection proposition: scope does not leak across the collection

Numbered profile: Graph-3

Required closure coordinate: Accountable provenance closure

Collection proposition: every reachable substrate is accountable

Numbered profile: Graph-4

Required closure coordinate: Refutable transformation closure

Collection proposition: challenges compose across hyperedges

Numbered profile: Graph-5

Required closure coordinate: Corroborated verification network

Collection proposition: member and edge witnesses are independently corroborated

A collection C satisfies candidate Graph profile N only if all four conditions hold:

1. **Membership floor** — every member rating is at object profile $\geq N$.
2. **Provenance closure** — every ancestor reachable from any member is rated at object profile $\geq N$.
3. **Status admissibility** — no ancestor is **REVOKED**, **CHALLENGED**, **STALE**, **UNKNOWN**, or subject to an unresolved **CONFLICTED** record.
4. **Edge evidence** — the transformation hyperedges themselves carry profile- N ratings.

Condition 2 is what a plain minimum over members misses. Condition 4 is what makes this dynamics rather than aggregation: **a graph is only as verified as its edges**, and an unevidenced edge between two profile-5 artifacts does not yield a Graph-5 collection.

The candidate Graph profile number is **computed from object and edge ratings, never declared**:

`candidate_graph_profile(C) = max { N : CNF_N(C) is satisfiable }`

The compatibility API `graph_level` implements that function and the frozen Graph receipt stores its number in a `level` field. The reference implementation searches 5->1 and certifies its Boolean encoding. Its current rating inputs are caller-supplied, so it reports a **candidate Graph profile** with `conformance_status = NOT_ESTABLISHED`; the certificate proves the computation over those inputs, not the validity of the ratings. At a result below 5, the grounded **FAIL** certificate for profile $N+1$ explains that candidate ceiling. Graph conformance additionally requires evidence-bound ratings under the applicable object and edge profiles. `establish_graph_level` supplies that path: it rehashes embedded evidence, reruns the exact registered rating mechanism for every member, ancestor, and reached edge, then recomputes and kernel-checks the Graph certificate. Each rating proposition binds a digest over the historical Graph bytes, deduplicated member set, collection identifier, and claim binding; neighboring collection or topology evidence therefore contributes zero. Missing, non-integer, or non-passing bindings also contribute zero and prevent conformance. Profile zero never receives **ESTABLISHED**. The record builder recomputes before serialization, and the rechecker preserves offline replay.

2.4 4. Why satisfiability is the spine

The decision procedure is not incidental to the design. Three consequences follow, and the third is the load-bearing one.

2.4.1 4.1 Some receipt fields are checkable certificates

VSTD does not classify every receipt as an NP certificate. Specific bounded formats, including VSTD4-GDC-1, define a finite decision problem, a certificate language, and a checker implemented separately from the producer path. Complexity claims apply only to such a defined formal problem; checker separation alone does not establish distinct actors. Other receipt fields may be signed declarations, hashes, measurements, or references whose meaning depends on explicitly named trust roots.

The useful engineering asymmetry is concrete rather than universal: when a result can carry a smaller consumer-checkable artifact instead of requiring the original computation, VSTD preserves that artifact and its verification bounds.

2.4.2 4.2 Bounded admission uses CNF

The reference admission procedures encode finite, bounded policy questions as [conjunctive normal form](#) (CNF) for the [Boolean satisfiability problem](#). CNF is not identical to 3-SAT. A finite CNF satisfiability instance can be transformed in polynomial time into an equisatisfiable 3-CNF instance, using auxiliary variables where required. VSTD does not need that transformation for every checker and does not infer a physical-world claim from the complexity of the encoded formula.

2.4.3 4.3 Global completeness is outside the observation boundary

PHYSICAL_WORLD_COMPLETENESS: UNSUPPORTED in VSTD-3 §16 means exactly that ordinary VSTD evidence does not enumerate every physical execution that could exist. That is an observational and claim-coordinate limit. It is not presented as a consequence of Tarski's theorem, as a generic co-NP classification, or as a proof that a future, explicitly finite observation system is impossible.

A future claim may cover a finite enumerated world if its observation boundary and completeness mechanism are declared and checked. It still MUST NOT be widened into a claim about unobserved physical activity.

This is why the object profile axis tops out at corroboration rather than proof of global absence. Profile 5 does not detect hidden work. It makes the *independence status* of declared work legible, and leaves the undeclared remainder named and quantified rather than silent.

2.5 5. Certificates for refusals

This section applies to the finite propositional decision procedures used by the reference Refutability implementation.

A satisfiable result already carries its certificate: the model. Anyone can evaluate it against the clause set without a solver.

An unsatisfiable result, by default, carries nothing but the solver's word.

For a fail-closed standard, **refusals are the most consequential output**. A standard whose passes are checkable and whose refusals are not has its assurance backwards. The Refutability coordinate therefore requires a refutation certificate — a clausal proof, verifiable by [reverse unit propagation](#), checkable without resolving. This follows the same producer-certificate/consumer-checker engineering asymmetry as [proof-carrying code](#), while using a narrower certificate language.

Resolution proofs have exponential lower bounds for some formula families. A conforming implementation therefore **MUST** declare a bound and **MUST** answer UNKNOWN when the implemented search or proof check exceeds that bound. **Fail-closed under resource exhaustion is a conformance requirement, not an implementation shortcut.** An UNKNOWN is never a pass and never an unsatisfiability claim.

Reference implementation: `verifier.core.refutation`.

2.5.1 5.1 The internal VSTD-4 rung sequence

VSTD-4 contains fourteen ordered rungs, from decision certification through semantic binding, anti-equivocation, bounded portable checking, availability, precommitment, challenge handling, degradation, and compositionality. Its normative depth is computed:

```
vstd4_depth(claim) = max { k : CNF_4k(claim) is satisfiable }
```

The certificate for rung $k+1$ explains a partial VSTD-4 normative depth. Only established VSTD-4 conformance at normative depth 14 admits a claim to any VSTD-5 procedure. The current reference `vstd4_depth` function instead computes a structural candidate from caller-supplied rung references, labels conformance NOT_ESTABLISHED, and never admits VSTD-5. `establish_vstd4` reruns exact VSTD-1/2/3 and rung bindings and may report EVIDENCE_BOUND / ESTABLISHED only when every mechanism and the independent kernel pass. See VSTD-4.md for the normative rung graph and VSTD4-GDC-1 format.

2.6 6. Composition — closure coordinates do not supply or substitute

Closure-coordinate results may be composed into an object profile-depth report. They do not replace, entail, or supply one another.

- Refutability without Substrate Accountability certifies a claim whose evidence source is unaccountable.
- Witness Corroboration without Refutability solicits witnesses for a claim no witness could check.
- Verification Surface without Claim Mechanics scopes a statement whose integrity is unestablished.

An implementation reporting object profile depth N **MUST** present separately checkable evidence for every required coordinate in profiles 1 through N . Conformance may also be reported for one coordinate without claiming cumulative profile depth. Incremental Substrate Accountability profiles are declared in VSTD-3 §7.

“Higher is more protected” is true only in the sense that more classes of failure are closed. It never means the prerequisite coordinates became unnecessary.

2.7 7. Numbering

- **Grounded certification obligations use X.M coordinates** within each object numbered profile; see **the obligation catalogue**.
- **Numbered profiles use integers:** VSTD-1 ... VSTD-5, VSTD-Graph-1 ... VSTD-Graph-5.
- **Repository releases use semantic versioning** and are independent of profile numbers.

A release version never implies a numbered profile, and a numbered profile never implies a release. See WIRE_IDENTIFIERS.md for serialized receipt identifiers and historical public filenames.

2.8 8. Intellectual lineage and adjacent precedents

The verification complex is VSTD project architecture; no cited work proves that these five coordinates on either axis are necessary, sufficient, complete, or uniquely ordered. The references below show that its individual design pressures have established precedents in security engineering, provenance, reproducible systems, and proof checking. The [concept guide](#) provides definitions, additional sources, and explicit non-equivalences.

VSTD pressure: Separate failure surfaces and fail-closed defaults

Adjacent precedent: Saltzer and Schroeder, *The Protection of Information in Computer Systems*

What the precedent contributes—and does not: Classic principles include fail-safe defaults, complete mediation, separation of privilege, and least common mechanism. They motivate separation; they do not derive VSTD’s coordinate count.

VSTD pressure: Named assurance components

Adjacent precedent: Common Criteria, [Part 3: Security assurance components](#)

What the precedent contributes—and does not: Demonstrates established componentized assurance and assurance packages. VSTD is not a Common Criteria evaluation or an Evaluation Assurance Level.

VSTD pressure: Stable cryptographic representations

Adjacent precedent: [RFC 8785: JSON Canonicalization Scheme](#)

What the precedent contributes—and does not: Shows why JSON used as cryptographic input needs invariant representation. VSTD formats retain their own declared canonicalization rules.

VSTD pressure: Recorded entities, activities, and agents

Adjacent precedent: W3C [PROV-DM](#)

What the precedent contributes—and does not: Supplies an interoperable provenance model adjacent to the Graph axis. VSTD-Graph is not a PROV implementation and does not infer complete history.

VSTD pressure: Software materials, builders, steps, and products

Adjacent precedent: [in-toto specification v1.0](#) and [SLSA v1.2](#)

What the precedent contributes—and does not: Establish supply-chain provenance and attestation precedents. VSTD may bind their evidence but cannot manufacture their authorization or assurance level.

VSTD pressure: Preserved release and provenance evidence

Adjacent precedent: NIST [Special Publication \(SP\) 800-218 SSDF 1.1](#)

What the precedent contributes—and does not: Protect the Software practices PS.3.1 and PS.3.2 call for preserving releases and provenance and enabling integrity verification. They do not certify a VSTD receipt.

VSTD pressure: Independent recreation

Adjacent precedent: Reproducible Builds, [formal definition](#)

What the precedent contributes—and does not: Grounds the special case where another party recreates specified artifacts from declared inputs and instructions. Reproducibility does not establish every semantic claim.

VSTD pressure: Producer-supplied portable certificates

Adjacent precedent: Necula, [Proof-Carrying Code](#)

What the precedent contributes—and does not: Establishes the pattern of an untrusted producer supplying a proof checked under a declared policy. VSTD uses the pattern beyond code safety without inheriting PCC’s theorem.

VSTD pressure: Consumer-checked UNSAT results

Adjacent precedent: Wetzler, Heule, and Hunt, [DRAT-trim](#)

What the precedent contributes—and does not: Establishes practical checking of clausal unsatisfiability proofs rather than trusting solver output. VSTD’s implemented RUP format is narrower than DRAT.

VSTD pressure: A first-class refusal to fabricate a Boolean answer

Adjacent precedent: [SMT-LIB Standard 2.7](#)

What the precedent contributes—and does not: Its response grammar includes **sat**, **unsat**, and **unknown**. VSTD independently defines a richer status system with the same fail-closed pressure.

VSTD pressure: Append-only public evidence and detectable equivocation

Adjacent precedent: [RFC 9162: Certificate Transparency Version 2.0](#)

What the precedent contributes—and does not: Merkle proofs make log inclusion and consistency auditable while preserving explicit split-view limitations. VSTD additive receipts are analogous, not a CT implementation.

VSTD pressure: Freshness, rollback, freeze, and compromise recovery

Adjacent precedent: [The Update Framework specification](#)

What the precedent contributes—and does not: Demonstrates that authentic old data is not automatically current data. VSTD does not implement TUF, but likewise keeps freshness and revocation distinct from byte identity.

Chapter 3

Grounded Certification Contracts and Mechanism Boundaries

Status: project specification for the additive version 1 certification contract. **Certificate identifier:** `verifier-grounded-certification-1`.

3.1 1. Coordinates and meaning

`X.M` identifies obligation `M` within object numbered profile `X`. These are dimensionless identifiers, not decimal numbers, software versions or confidence scores. Each profile has its own finite sequence. `VSTD-4` retains the exact fourteen rungs and dependency graph from `VSTD-4.md`; other entries are called obligations. The Graph axis has no new dotted coordinates in this contract.

Grounded certification requires the exact claim, artifact coordinate, obligation, evidence bytes, mechanism implementation, trust roots and bounds to be bound and checked. Certificates carry replay inputs and computed results. They **MUST NOT** accept a declarant-supplied success, profile depth or certification grade as input.

The normative tables below decompose the existing object-profile requirements. They do not replace the full requirements in `VSTD-1.md` through `VSTD-5.md`. A domain mechanism **MUST** evaluate the stated obligation inside the exact declared scope. Merely checking a record's shape, repeating its fields or matching a digest does not discharge a substantive obligation about the record's meaning.

3.2 2. Admission and grounding

The checker **MUST** supply an external `verifier-grounded-policy-1` admission policy. For each admitted obligation it selects an exact mechanism identifier, implementation digest and trust-root set. The policy pins the obligation catalogue and normative specification bytes. Registration and policy admission declare a trust boundary; they do not establish that a mechanism is correct or independent. Domain-specific mechanism qualification remains necessary before relying on its conclusions.

Each `verifier-grounded-request-1` binds a claim identifier, complete claim commitment, target numbered profile and an individually bound proposition for each supplied obligation. The claim's policy root **MUST** equal the external policy digest. Its evidence root **MUST** commit to the sorted unique set of all referenced evidence addresses. Every obligation binds the same claim identifier and commitment, its exact catalogue predicate, Boolean true, admitted mechanism, trust roots and input bounds. Claims about two different identifiers require separate checking; no alias is inferred. Unknown or out-of-target obligation identifiers **MUST** be rejected.

Rechecking **MUST** be anchored to an externally expected request digest. A certificate cannot establish that its claim is the one a consumer intended to check merely by carrying its own claim identifier. A checker digest binds the reference orchestration implementation; each domain mechanism remains separately pinned.

3.3 3. Evaluation and dependency semantics

Every supplied admissible obligation **MUST** have its evidence rehashed and its mechanism rerun. The direct outcome is **PASS**, **FAIL** or **UNKNOWN**. Missing evidence, unavailable mechanisms, digest mismatch, unapproved trust roots, unsupported checking and exhausted input bounds **MUST NOT** become success. Invalid embedded byte bindings **MUST** be rejected. A mechanism refutation remains **FAIL**, even if another obligation is **UNKNOWN**.

An obligation is established only if its direct outcome is **PASS** and all its listed dependencies are established. A blocked obligation retains its direct observation separately. A profile coordinate is established only when all its obligations are established. A cumulative profile additionally requires every earlier profile coordinate. The `certified_profile_depth` is the largest uninterrupted established prefix, or zero; it never suppresses later direct evidence. Per-profile `established_prefix` counts contiguous established obligations within that profile. Neither count is a strength score.

Overall **PASS** means all required obligations through the requested profile are established under the admitted mechanisms, evidence and limits. Any directly failed obligation produces overall **FAIL**; otherwise incomplete support produces **UNKNOWN**. `certification_status` is **ESTABLISHED** only for the complete requested prefix. A failed certification obligation does not imply a universal claim about the artifact.

An inapplicable domain requirement is not silently omitted. A mechanism must check the precise applicability boundary and show that the scoped obligation is satisfied. For VSTD-3 this preserves declared capability limits: discovery is never promoted to attestation, complete mediation or physical-world completeness. VSTD-5 requires actual corroboration and all seven evidenced separation dimensions. Names, hashes, duplicate observations and witness counts never manufacture independence.

3.4 4. Portability, bounds and compatibility

A certificate embeds all available referenced evidence bytes and preserves absent references in the request so an **UNKNOWN** result can replay. It carries the exact request, catalogue/specification/policy/checker digests and complete derived result. Its canonical digest establishes byte integrity only. Rechecking **MUST** validate the exact shape, externally expected request, external policy and implementation coordinate, rehash evidence, rerun the mechanisms, and compare the complete derived result. Neither deleting a blocker nor replacing the carried result may survive replay.

Input counts are dimensionless; evidence and certificate limits are bytes. Global evidence limits apply before domain execution; each proposition retains its tighter input limits. The reference parser also imposes a 32 mebibyte (MiB, 1,048,576 bytes per unit) document ceiling. The reference engine does not sandbox or interrupt arbitrary Python mechanism code; deployments must admit bounded checkers or isolate them. Input byte ceilings are not a claim of bounded arbitrary computation or physical resource enforcement.

The new request, policy and certificate identifiers are additive. Existing receipt identifiers, VSTD-4 rung predicates, candidate-depth behavior, native VSTD-5 entry checks and historical receipts remain unchanged. A legacy receipt is evidence for a suitable mechanism, never automatically a complete new certificate. The new engine does not convert a certificate into a legacy VSTD-4 or VSTD-5 receipt.

Evidence degrades when the claim, artifact, policy, mechanism, trust roots, bounds, specification or dependency evidence changes. Old receipts remain historical bytes; a changed coordinate requires a new assessment. Portable replay establishes only the registered mechanisms' scoped propositions, not external independence, accreditation, physical containment or correctness of unimplemented domain checks.

3.5 5. Object-profile obligation catalogue

The reference catalogue is `verifier.core.profile_obligations.OBLIGATIONS`. Every row below is individually evidence-bound. Dependencies listed here are within the same numbered profile; cumulative profile prerequisites also apply.

3.5.1 VSTD-1: Claim Mechanics

Obligation: 1.1

Name: Claim coordinate

Checked proposition: The exact subject, proposition, scope, limitations and falsifier are bound.

Dependencies: none

Source: VSTD-1.md sections 2 and 4

Obligation: 1.2

Name: Evidence binding

Checked proposition: Every verdict-material evidence item is identified, available and bound to this claim.

Dependencies: 1.1

Source: VSTD-1.md section 7

Obligation: 1.3

Name: Checker and trust boundary

Checked proposition: The actual checker implementation, dependencies and achieved role separation are evidenced without inferring independence.

Dependencies: 1.1, 1.2

Source: VSTD-1.md section 5

Obligation: 1.4

Name: Decision replay

Checked proposition: The declared decision follows from rerunning the bound computational checker on the bound evidence within its limits.

Dependencies: 1.2, 1.3

Source: VSTD-1.md sections 2, 3 and 5

Obligation: 1.5

Name: Provenance

Checked proposition: Source and execution provenance match the observed evidence and their stated observation boundary.

Dependencies: 1.2

Source: VSTD-1.md section 7

Obligation: 1.6

Name: Reproduction fidelity

Checked proposition: The declared reproduction result and fidelity are supported by a checked comparison of the named executions.

Dependencies: 1.4, 1.5

Source: VSTD-1.md section 6

Obligation: 1.7

Name: Challenge and correction

Checked proposition: The falsification procedure is executable and any observed challenge or correction is retained without rewriting historical evidence.

Dependencies: 1.1, 1.4

Source: VSTD-1.md section 8

3.5.2 VSTD-2: Verification Surface

Obligation: 2.1

Name: Subject and surface

Checked proposition: The primary subject, selected surface, exclusions and coordinate meanings are explicit and bound.

Dependencies: none

Source: VSTD-2.md section 3

Obligation: 2.2

Name: Geometry consistency

Checked proposition: All selected coordinate references, containment relations and seams are internally consistent.

Dependencies: 2.1

Source: VSTD-2.md sections 3 and 10

Obligation: 2.3

Name: Translation and reconstruction

Checked proposition: The named translation or reconstruction is checked against its source and exposes information loss and unresolved mappings.

Dependencies: 2.1, 2.2

Source: VSTD-2.md sections 4 and 8

Obligation: 2.4

Name: Evidence-earned judgments

Checked proposition: Every relied-on judgment is earned by its exact evidence and executed mechanism rather than a status label.

Dependencies: 2.1, 2.2

Source: VSTD-2.md sections 2 and 10

Obligation: 2.5

Name: Residuals and horizons

Checked proposition: Residuals, exclusions and horizons are typed, localized and retained in the assessed scope.

Dependencies: 2.2, 2.3

Source: VSTD-2.md sections 5 and 6

Obligation: 2.6

Name: Adjacent verification orders

Checked proposition: Each represented meta-verification order checks its immediate predecessor and retains its evidence and termination horizon.

Dependencies: 2.2, 2.4

Source: VSTD-2.md section 6.4

Obligation: 2.7

Name: Bounded surface closure

Checked proposition: The declared closure result is recomputed for the exact selected surface with every blocker preserved.

Dependencies: 2.3, 2.4, 2.5, 2.6

Source: VSTD-2.md sections 6 and 10

3.5.3 VSTD-3: Substrate Accountability

Obligation: 3.1

Name: Subject and capability boundary

Checked proposition: Observed devices, execution scope, capability class and unsupported capabilities match the declared substrate boundary.

Dependencies: none

Source: VSTD-3.md sections 3, 5, 6 and 7

Obligation: 3.2

Name: Attestation binding

Checked proposition: Every relied-on attestation binds its challenge, subject, evidence and declared trust root; absent authentication cannot support an attested claim.

Dependencies: 3.1

Source: VSTD-3.md section 8

Obligation: 3.3

Name: Firmware accountability

Checked proposition: Firmware identity, measurement and accountability claims are checked under the declared firmware contract and capability boundary.

Dependencies: 3.1, 3.2

Source: VSTD-3.md section 9

Obligation: 3.4

Name: Execution binding

Checked proposition: Every claimed execution is linked to the exact workload, inputs, outputs and evidenced execution boundary.

Dependencies: 3.1, 3.2

Source: VSTD-3.md section 12

Obligation: 3.5

Name: Accounting and topology

Checked proposition: Reported accounting and partition/topology claims are recomputed from bound evidence with uncertainty and virtualization limits preserved.

Dependencies: 3.1, 3.4

Source: VSTD-3.md sections 13 and 14

Obligation: 3.6

Name: Continuity and anchors

Checked proposition: Claimed continuity and external anchors are checked for the declared interval without inferring unobserved continuity.

Dependencies: 3.2, 3.3, 3.4

Source: VSTD-3.md sections 10 and 11

Obligation: 3.7

Name: Provider and fleet scope

Checked proposition: Provider and fleet statements are checked only inside their enumerated evidence boundary; unobserved physical work remains unsupported.

Dependencies: 3.1, 3.4, 3.5

Source: VSTD-3.md sections 15 and 16

Obligation: 3.8

Name: Derived substrate outcome

Checked proposition: The substrate outcome is recomputed from the preceding obligations and capability-specific evidence without promoting an unsupported claim.

Dependencies: 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7

Source: VSTD-3.md sections 17, 20 and 24

3.5.4 VSTD-4: Refutability

Obligation: 4.1

Name: Decision certification

Checked proposition: PASS, FAIL and UNKNOWN each carry a checkable artifact

Dependencies: none

Source: VSTD-4.md section 2

Obligation: 4.2

Name: Semantic binding

Checked proposition: the certificate proves the exact declared claim coordinate

Dependencies: 4.1

Source: VSTD-4.md section 2

Obligation: 4.3

Name: Anti-equivocation

Checked proposition: claim, coordinate, policy, evidence, verifier, bounds and prior commitment bind into one digest C

Dependencies: 4.2

Source: VSTD-4.md section 2

Obligation: 4.4

Name: Portable verification

Checked proposition: checking needs no post-verdict cooperation from the declarant

Dependencies: 4.3

Source: VSTD-4.md section 2

Obligation: 4.5

Name: Bounded verification

Checked proposition: declared cost, memory and size bounds, enforced by the checker on itself

Dependencies: 4.4

Source: VSTD-4.md section 2

Obligation: 4.6

Name: Re-derivability

Checked proposition: no undeclared hidden state, unpinned dependency, local path, wall-clock read or ambient entropy is verdict-material

Dependencies: 4.4

Source: VSTD-4.md section 2

Obligation: 4.7

Name: Minimal trusted checker

Checked proposition: certificate semantics implementable with zero shared verdict-producing code

Dependencies: 4.5

Source: VSTD-4.md section 2

Obligation: 4.8

Name: Availability

Checked proposition: verdict-critical artifacts are AVAILABLE, PORTABLE or SELF_CONTAINED, not merely IDENTIFIED

Dependencies: 4.6

Source: VSTD-4.md section 2

Obligation: 4.9

Name: Disclosure-safe checkability

Checked proposition: confidential evidence still satisfies a declared verification interface

Dependencies: 4.8

Source: VSTD-4.md section 2

Obligation: 4.10

Name: Explicit refutation surface

Checked proposition: machine-readable admissible_refutations and excluded_claims

Dependencies: 4.2

Source: VSTD-4.md section 2

Obligation: 4.11

Name: Prior commitment

Checked proposition: a PrecommitmentEnvelope over every verdict-material degree of freedom

Dependencies: 4.10

Source: VSTD-4.md section 2

Obligation: 4.12

Name: Challenge handling

Checked proposition: valid counterevidence deterministically changes claim status

Dependencies: 4.10, 4.1

Source: VSTD-4.md section 2

Obligation: 4.13

Name: Monotonic degradation

Checked proposition: weakening evidence can never preserve an unsupported verdict

Dependencies: 4.12, 4.8

Source: VSTD-4.md section 2

Obligation: 4.14

Name: Compositionality

Checked proposition: a RefutabilityClosure states how refutability propagates through transformations

Dependencies: 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 4.10, 4.11, 4.12, 4.13

Source: VSTD-4.md section 2

3.5.5 VSTD-5: Witness Corroboration

Obligation: 5.1

Name: Exact refutability entry

Checked proposition: The exact claim has evidence-bound prerequisite profiles and all fourteen VSTD-4 rungs; a candidate depth cannot admit a witness.

Dependencies: none

Source: VSTD-5.md section 1

Obligation: 5.2

Name: Witness identity binding

Checked proposition: Each witness identity is bound to available identity evidence; duplicate evidence and dangling identities do not create witnesses.

Dependencies: 5.1

Source: VSTD-5.md sections 2 and 3

Obligation: 5.3

Name: Operational separation

Checked proposition: Ownership or operational control separation is checked for the exact declarant/witness pair and claim commitment.

Dependencies: 5.2

Source: VSTD-5.md section 3

Obligation: 5.4

Name: Implementation separation

Checked proposition: Separation of verdict-producing code is checked for the exact declarant/witness pair and claim commitment.

Dependencies: 5.2

Source: VSTD-5.md section 3

Obligation: 5.5

Name: Trust-root separation

Checked proposition: Verifier trust-root separation is checked for the exact declarant/witness pair and claim commitment.

Dependencies: 5.2

Source: VSTD-5.md section 3

Obligation: 5.6

Name: Evidence-source separation

Checked proposition: Evidence-source or telemetry-provider separation is checked for the exact declarant/witness pair and claim commitment.

Dependencies: 5.2

Source: VSTD-5.md section 3

Obligation: 5.7

Name: Infrastructure separation

Checked proposition: Separation of infrastructure capable of changing the result is checked for the exact declarant/witness pair and claim commitment.

Dependencies: 5.2

Source: VSTD-5.md section 3

Obligation: 5.8

Name: Financial separation

Checked proposition: Material financial dependence is checked for the exact declarant/witness pair, corroboration and claim commitment.

Dependencies: 5.2

Source: VSTD-5.md section 3

Obligation: 5.9

Name: Compulsion separation

Checked proposition: Material jurisdictional or contractual dependence is checked for the exact declarant/witness pair and claim commitment.

Dependencies: 5.2

Source: VSTD-5.md section 3

Obligation: 5.10

Name: Executed corroboration

Checked proposition: Actual witness checking binds the admitted certificate, checker, class, observation time, evidence and result; declarations alone do not satisfy it.

Dependencies: 5.1, 5.2, 5.3, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9

Source: VSTD-5.md section 4

Obligation: 5.11

Name: Disagreement preservation

Checked proposition: All checked disagreements and duplicate-evidence findings are retained; voting or repeated evidence cannot produce independent corroboration.

Dependencies: 5.10

Source: VSTD-5.md sections 4 and 5

Chapter 4

Version 2 Candidate Scope and Release Boundaries

Verifier Standard (VSTD) 2.0.0 is **unreleased**. The Python Package Index (PyPI) and GitHub release list were checked on 2026-09-19: the latest published package was 1.5.0. Recheck those services before publication; this observation expires when either changes. Install a candidate from its reviewed source checkout.

4.1 Release name and grounded certification

Verifier / verifier-standard v2.0.0 — The grounded certification release. This release name was selected by the maintainer on 2026-09-19.

Grounded certification means that a decision certificate binds an exact claim and artifact coordinate to its evidence, encoding rules, checking mechanism, and declared bounds. Its grounding must be checked; a label or digest alone does not establish the claimed proposition. These mechanisms predate this candidate's experimental additions. The existing grounded decision certificate (GDC) format **VSTD4-GDC-1** and checker support bounded decision checking; kernel acceptance alone does not establish full VSTD-4 conformance. The evidence-bound **establish_vstd4** path separately reruns mechanisms for prerequisite profiles and all fourteen rungs.

The release now adds 47 individually evidence-bound obligation coordinates across all five object numbered profiles: 1.1--1.7, 2.1--2.7, 3.1--3.8, 4.1--4.14, and 5.1--5.11. VSTD-4 retains its existing rung meanings; the other coordinates are obligations, not software versions or certification strength scores. The VSTD-3.0 receipt identifier remains unchanged. See the **normative contract** and **executable certification guide**.

The new engine binds an external admission policy, checks each exact proposition, retains missing prerequisites and refutations, and emits portable replayable certificates. Six obligations have built-in native adapters; the remaining 41 require explicitly registered domain mechanisms that check their stated propositions. The catalogue and orchestration do not by themselves discharge those obligations.

The candidate also adds nine native grounded domain adapters with 43 cumulative checks: dataset transformations and split/overlap analysis; retained environment inventory/configuration/resource/reproduction observations; executable benchmark oracles and scoring; checkpoint/optimizer/gradient replay; model inference, metrics and challenges; simulation transition/invariant/projection/channel/shard checks; declared agent observation surfaces and their retained transcripts; agent trajectories bounded by one harness certificate's observation ceiling; and the closed loop between one bound agent certificate and one bound simulation certificate. These paths have portable certificates with external request and policy binding. Their supported formats and exclusions are defined in **the domain contract**. A complete native domain assessment is not complete object-profile certification.

4.2 Recovered scope and lifecycle

This candidate includes the grounded certification engine and commands, the pending `vstd publish` command, optional grammar constraint testing, and experimental `verifier.corrigibility` modules. The source version is a packaging coordinate, not evidence that new verification mechanisms or normative profiles are complete. Existing **standard/** specifications remain authoritative. The draft schemas are record formats, not conformance mechanisms.

The following table describes the retained experimental helpers, whose historical receipts are not upgraded by the new grounded domain paths above.

Experimental component: Graph

Implemented boundary: Directed acyclic graph structure, declared status and dependency checks

Not established: Truth of supplied premises or independent proof checking

Experimental component: Environment

Implemented boundary: Declared hardware/software records, comparisons and consistency rules

Not established: Network isolation, resource enforcement or authenticated observation

Experimental component: Benchmark

Implemented boundary: Problem records, declared receipt aggregation and timeout rules

Not established: Problem-bound oracle execution; a validity flag yields UNKNOWN

Experimental component: Dataset

Implemented boundary: Digest/Merkle records and lexical overlap checks on supplied strings

Not established: Source authenticity, complete contamination detection or embedding analysis

Experimental component: Training/model

Implemented boundary: Declared checkpoint lineage, curriculum fields and score comparisons

Not established: Reproduced training, calibrated risk scores, alignment or corrigibility

Experimental component: Tesla cage

Implemented boundary: Declared hardware prerequisites and command preflight

Not established: Physical containment, hardware authenticity, shutdown execution or tool unboxing

Experimental component: Simulation 1

Implemented boundary: Replay comparison of supplied canonical state records

Not established: General determinism beyond this execution and input

Experimental component: Simulation 2

Implemented boundary: Predicates evaluated on supplied finite states

Not established: Inductive invariance over unobserved states

Experimental component: Simulation 3

Implemented boundary: Projected trace distance under a caller-provided metric

Not established: Universal bisimulation or a proof about transition operators

Experimental component: Simulation 4

Implemented boundary: Supplied observations and numeric actions checked against declared bounds

Not established: Runtime isolation, complete channel capture or prevention of leakage

Experimental component: Simulation 5

Implemented boundary: Supplied shard consistency diagnostics

Not established: Witness authentication, independent nodes or distributed consensus

The simulation checks are cumulative in the evaluator. Missing prerequisite mechanisms yield `NOT_ESTABLISHED`; observed violations retain `VIOLATION_DETECTED`. `VERIFIED` is limited to the local supplied-trace checks, not existing numbered-profile conformance. Callable identities and their execution environments are not bound in these draft receipts, so a receipt alone is not portable proof that a particular program was checked. Distances and epsilon bounds have the units selected by the caller's metric; no physical units are inferred.

Hardware record digests establish record identity only. `TeslaCageSandbox` does not sandbox a process. `execute_preemption` raises `NotImplementedError` until an actual backend and observation are implemented. Boolean declarations never admit tool unboxing. Risk aggregation never emits `CONTAINED`; its default is `UNVERIFIED`. Its other verdicts describe declared-input consistency findings, not observed physical breaches. Risk thresholds are draft policy constants with no demonstrated empirical calibration. Refutation records carry caller-reported outcomes; no response predicate is executed by the current model helper.

4.3 Compatibility and migration

- The distribution remains `verifier-standard`, imported as `verifier`. Commands `vstd`, `verifier`, and `verifiable` retain their existing entry point.
- No existing numbered-profile wire identifier is renamed by this candidate. Experimental identifiers under `verifier.corrigibility` are not aliases for normative VSTD object or Graph profiles.
- `Vstd160RiskProfileReceipt` and `evaluate_vstd160_risk_profile` are aliases for historical local draft callers only. Acceptance of draft version strings does not establish compatibility with any published 1.5.0 or 1.6.0 risk format.
- Callers must handle `UNKNOWN`, `UNVERIFIED`, `NOT_ESTABLISHED` and unsupported operation exceptions. Code relying on self-reported success must migrate to independently checked evidence before making stronger claims.
- Physical containment still requires a separate mechanism and validation gate. Portable finite simulation computation is available through the grounded domain path; the earlier experimental receipts do not acquire that binding retroactively.

4.4 Publication gates

Finalize scope and review the exact candidate diff, commit through the signed contribution workflow, run the full hosted matrix on that commit, and satisfy the current human-acceptance policy. Then follow **the release procedure** for deterministic artifact comparisons, installed-wheel checks, boundary scanning, actual-date finalization, and separately authorized publication. No tag, merge, upload, or deployment is implied by local preparation.

Chapter 5

Formal System Architecture and Kernel Isolation

Acronyms: application programming interface (API); Boolean satisfiability problem (SAT); command-line interface (CLI); identifier (ID); JavaScript Object Notation (JSON); reduced instruction set computer (RISC); Supply Chain Integrity, Transparency, and Trust (SCITT); Verifier Standard (VSTD); zero-identity/zero-knowledge (ZIZK).

Status: implementation and ownership map; normative meaning remains in **standard/**

TRUST is mechanism-earned forward artifact support; ROT is typed, time-indexed degradation of current admissibility; and RUST is inverse-TRUST diagnostic traversal toward recorded ancestors. They are formal semantic names, not acronyms or actor ratings.

Use this order when two surfaces appear to disagree:

1. normative numbered-profile document;
2. serialized receipt identifier (`schema_version`) and profile discriminator;
3. published JSON Schema;
4. typed model and validator;
5. conformance tests;
6. generated reference and examples.

A lower item cannot silently redefine a higher item. A passing schema check establishes shape only; a passing validator establishes only its named implemented checks.

5.1 Numbered-profile ownership

Coordinate: VSTD-1 claim receipt

Normative source: `standard/VSTD-1.md`

Runtime owner: `verifier.core.receipt`, `verifier.core.checker`

Published shape: `vstd1_receipt.json`

Primary tests: `test_independent_checker.py`, `test_vstd_schemas.py`

Coordinate: VSTD-1 generic run

Normative source: `standard/VSTD-1.md`

Runtime owner: `verifier.core.run` capture/facade plus `run_planning`, `run_validation`, `run_inspection`, `run_reproduction`, `run_impact`, and the additive `platform_comparison` diagnostic

Published shape: `vstd1_generic_run_receipt.json`; the comparison report is not a receipt

Primary tests: `test_generic_run.py`, `test_platform_comparison.py`

Coordinate: VSTD-2

Normative source: `standard/VSTD-2.md`

Runtime owner: `verifier.core.geometry`

Published shape: `vstd2_receipt.json`

Primary tests: `test_verification_geometry.py`

Coordinate: VSTD-3

Normative source: `standard/VSTD-3.md`

Runtime owner: `verifier.hardware`

Published shape: `vstd3_receipt.json`, `vstd3_accelerator_profile.json`

Primary tests: `test_vstd3_schema.py`, hardware tests

Coordinate: VSTD-4

Normative source: `standard/VSTD-4.md`

Runtime owner: `certificate/kernel` checks plus candidate and evidence-bound paths in `verifier.core.depth` / `verifier.core.evidence`

Published shape: `vstd4_certificate.json`, `vstd4_receipt.json`

Primary tests: `test_gdc_certificate.py`, `test_vstd4_depth.py`, `test_evidence_bound_assurance.py`

Coordinate: VSTD-5

Normative source: `standard/VSTD-5.md`

Runtime owner: `verifier.core.witness` evidence-bound entry, independence, corroboration, disagreement, build, and replay

Published shape: `vstd5_receipt.json`

Primary tests: `test_evidence_bound_assurance.py`, `test_vstd_schemas.py`

Coordinate: VSTD-Graph-1

Normative source: `standard/VSTD-Graph-1.md`

Runtime owner: `verifier.data.models`, `verifier.data.receipt`

Published shape: `vstd_graph_receipt.json`

Primary tests: `test_public_data.py`

Coordinate: VSTD-Graph-2..5

Normative source: matching Graph documents

Runtime owner: `verifier.data.graph_level` candidate/evidence-bound paths

Published shape: `computed_graph_level` within `vstd_graph_receipt.json`

Primary tests: `test_graph_level.py`, `test_evidence_bound_assurance.py`

Coordinate: ZIZK artifact-first TRUST/ROT/RUST

Normative source: `standard/LADDER.md` section 1.1

Runtime owner: `verifier.data.assurance`; bounded RISC Zero example under `examples/zizk_artifact_first/`

Published shape: `verifier-graph-assurance-1.schema.json`; not a numbered-profile receipt

Primary tests: `assurance`, `presentation`, `experiment-manifest`, and ZIZK mechanism tests

Coordinate: Artifact freeze, seal, and thaw

Normative source: `standard/ARTIFACT_CONTROL.md`

Runtime owner: `verifier.artifact_control` and `vstd artifact`

Published shape: `standard/schemas/artifact-control-1.schema.json`; these are mechanism objects, not receipts

Primary tests: `test_artifact_control.py`, public API/CLI tests

Compatibility VSTD-4 and Graph paths still compute candidates from caller-supplied references or ratings and return `conformance_status = NOT_ESTABLISHED`. Separate evidence-bound paths resolve exact bytes, pin and rerun registered mechanisms, enforce bounds, and recheck certificates before they can report `ESTABLISHED`. A mechanism result is limited to its proposition, evidence, trust roots, implementation digest, and bounds; the repository claims no external witness or independent implementation.

Artifact control is an orthogonal mechanism beneath the axes. It can preserve and close an artifact used in any numbered profile, but its successful verification establishes only exact-byte integrity, structural closure, and any separately supplied external anchor. It cannot supply a numbered-profile result, semantic correctness, encryption, trusted time, actor trust, or continuous temporal mediation. A thaw sidecar is additive lineage metadata, not parent authentication: sidecar-only agreement remains `NOT_ESTABLISHED`. Current clean or dirty status requires an actual supplied, cleanly sealed parent whose artifact, content, freeze, seal, kind, and media-type coordinates match. Even that comparison does not authenticate the historical copy operation. Creation paths preserve the caller-supplied final entry through classification: freeze refuses symbolic-link sources, while bundle, descendant, and sidecar creation require an absent lexical destination. These checks do not claim universal race-free filesystem security against concurrent privileged replacement. Authoritative members inside a bundle are stricter: the freeze manifest, payload, seals container, and seal envelopes must have their required ordinary lexical types and cannot borrow bytes through symbolic links or supported reparse-point aliases. That internal closure rule does not reject an outer parent-bundle or explicit thaw-record read alias, whose resolved bytes and bindings are verified. Ordinary hard links remain byte-and-path objects rather than claims of exclusive inode ownership.

5.2 Verification complex and profile satisfaction

VSTD is organized by named **closure coordinates**, not interchangeable layers or scalar assurance levels. A **numbered profile** is a cumulative requirement formula over those coordinates. The two axes use different coordinate sets even when their profile numbers match:

Profile number: 1

Object closure coordinate: Claim Mechanics

Graph closure coordinate: Recorded Lineage

Profile number: 2

Object closure coordinate: Verification Surface

Graph closure coordinate: Bounded Collection Surface

Profile number: 3

Object closure coordinate: Substrate Accountability

Graph closure coordinate: Accountable Provenance Closure

Profile number: 4

Object closure coordinate: Refutability

Graph closure coordinate: Refutable Transformation Closure

Profile number: 5

Object closure coordinate: Witness Corroboration

Graph closure coordinate: Corroborated Verification Network

Evidence enters through a named mechanism, establishes or fails to establish exact coordinate facts, and is then evaluated against the selected profile formula. A complete assessment preserves **ESTABLISHED**, **REFUTED**, **UNKNOWN**, **CONFLICTED**, and **NOT_ESTABLISHED** rather than collapsing absent evidence into false or a satisfiable encoding into conformance. A satisfying assignment over unvalidated caller assertions is only a candidate. Satisfaction suffices only when every satisfying fact is itself bound to evidence by the mechanism that earned it.

“Closure” is always qualified. VSTD-2 surface closure, Graph provenance closure, refutability closure, and artifact-seal structural closure are distinct propositions. “Profile” is likewise qualified as a numbered, receipt, application, or geometry profile when context does not make the category unique. The exact compatibility names and exceptions are normative in [standard/LADDER.md](#).

5.3 Operational traversal and recursive Graph materialization

This non-serialized implementation view does not redefine the numbered profiles:

```
native computation
-> VSTD-1 execution and claim capture
-> profiler or domain adapter
-> VSTD-2 verification-surface normalization
```


- > VSTD-3 substrate accountability
- > VSTD-4 portable refutation
- > VSTD-5 independently evidenced witness corroboration
- > VSTD-Graph collection assessment
- > content-addressed result artifact
- > later bounded verification loop

5.3.1 Experimental interoperability planning side path

Version 1.3.0 also exposes a deliberately nonexecuting planning side path:

```
strict VSTD-2 JSON document
  -> strict wire loader and semantic validation
  -> typed valid geometry
  -> modeled-hole analysis
  -> exact catalog candidates
  -> nonexecuting validation plan
```

`load_verification_geometry` strictly accepts a VSTD-2 JSON file or parsed mapping, rejects unknown structure and invalid references, and returns only a semantically valid typed `VerificationGeometry`. `analyze_verification_surface` accepts that typed object, revalidates its structure, assesses its declared ordinary and self-closure state, and converts existing blockers into typed holes. The public `vstd surface analyze` command joins those two supported operations. Neither operation infers an omitted expected profile, ontology, coordinate, or real-world surface, and successful loading establishes only the declared VSTD-2 structure—not the truth or completeness of its contents.

`plan_validation` connects those holes to immutable component descriptors through exact schema and interaction-mode coordinates plus the hole’s relation and mechanism coordinates where present. Domain tags, consequence profiles, and operating-regime text remain organization and planning metadata; they cannot manufacture a match. A plan binds the geometry digest plus registry version and digest, retains execution prerequisites and blockers, and fixes `plan_only` to true and `execution_performed` to false.

In this architecture, **validation** is the future process of attempting to discharge holes with authorized component execution and bound evidence, followed by a fresh surface analysis. Candidate matching alone does not establish component availability at execution time, checker correctness, a native result, ordinary or self-closure, safety, authority, critical-domain readiness, or VSTD conformance. The runnable **sorted-grocery-list example** intentionally leaves three self-closure requirements unmatched and never calls its checker.

The **artifact-awareness contract** separates actual awareness, permitted awareness and potentially inferable awareness between artifacts. Confidentiality of awareness constrains exposure and propagation; authorization and catalog matching do not establish it. Version 1.3.0 adds an experimental exact-graph-bound analyzer for finite, observer-relative knowledge-hypergraph closure across composed interfaces. It does not track awareness continuously, establish open-world inference completeness, or enforce confidentiality at runtime.

The separate **experimental graph-topology interpretation** binds existing graph artifacts, transformations and ports to explicit simultaneous Boolean equations or clock-relative temporal relations. Structural cycles, encoded consistency, and mechanism-earned support remain distinct. `vstd data topology` runs this bounded diagnostic; catalog discovery and planning do not. Existing cyclic-assurance admission guards and receipt representations remain unchanged.

VSTD-2 is the semantic target for adjacent adapters, not the adapter implementation itself. Geometry profiles constrain reusable selections of VSTD-2 geometry; they are connected only by explicit shared coordinates, seams, mappings, and evidence-bearing transformations. The current VSTD-2 receipt has no geometry-profile or profile-composition field, so this relationship is conceptual rather than a new serialized contract. See VSTD-2 section 8.1.

The complete apparatus that constructs or assesses a Graph is a verifying process. In a later order it may

become the subject of a new VSTD-2 surface, while an adjacent adapter maps its selected observable outputs into that geometry. Treating the whole apparatus as the adapter would erase the builder, mapping, verifier, and output seams that the next assessment must examine.

A materialized Graph result must retain or bind the source Graph receipt, target collection or induced subgraph, selection query, object and edge ratings with their evidence, selected surface, lifecycle and conflict state, Graph profile certificate, materialization mechanism, and declared information loss. It earns no strength from size, path count, repetition, storage, or agreement. Its result is capped by every applicable member, transformation, mapping, substrate, refutation, witness, and materialization obligation. The compatibility path establishes only its candidate computation. The evidence-bound path can establish a Graph profile only after rerunning every required rating mechanism across the complete closure.

5.4 Governing ZIZK architecture and mechanism ownership

ZIZK artifact-first TRUST is a governing VSTD architecture, not a side experiment, numbered profile, scalar trust system, or actor reputation system. VSTD evaluates bounded validity propositions about computational processes represented by software and evidence-bearing artifacts; it does not determine whether an actor is good or bad. Its normative source is `standard/LADDER.md` section 1.1.

Zero identity means zero identity-derived verdict weight, not anonymity or absence of identifiers. Checked identity evidence may establish only its exact attribution, authorization, or separation proposition, adjacent to the process claim. Zero knowledge means zero unevidenced knowledge is presumed: absent a mechanism-earned result, the exact proposition remains UNKNOWN. When a witness must remain confidential, cryptographic zero knowledge may enclose this architectural rule by binding the exact program, predicate, public commitments, output, proof parameters, and verifier. The proof remains bearer- and artifact-bound; prover identity or reputation supplies no TRUST. This property applies only where a named proof system establishes it under explicit assumptions.

TRUST, ROT, and RUST are formal semantic names, not acronyms, serialized receipt values, actor ratings, scalar scores, or references to the Rust programming language:

- TRUST is mechanism-earned artifact support moving forward only across admissible bound transformations while every child discharges its new obligations;
- ROT is typed, time-indexed degradation of current admissibility without rewriting historical evidence; and
- RUST is the inverse-TRUST diagnostic mechanic moving backward from an observed descendant deviation through recorded contributing ancestry.

The three never cancel, reverse direction, or manufacture a clean signal from UNKNOWN or CONFLICTED inputs. Identity, popularity, repetition, age alone, topology, and propagation supply no assurance.

Maturity attaches to mechanisms beneath that architecture:

Mechanism surface: RISC Zero hidden-witness predicate

Current status: Bounded reference mechanism with tracked public proof artifacts and a governed tracked-source-build/image-ID equality gate

Ownership boundary: `examples/zizk_artifact_first/risc0/`; native verification only, no VSTD receipt mapping or independent build host

Mechanism surface: Bounded identity-disclosure evaluator

Current status: Bounded non-normative reference mechanism

Ownership boundary: `examples/zizk_artifact_first/zero_identity/`; no identity-derived trust

Mechanism surface: Assurance event serialization and replay

Current status: Implemented bounded reference mechanism

Ownership boundary: `verifier-graph-assurance-1` embeds the historical Graph, exact bindings, evidence bytes, a hash chain, and a current-view digest; `recheck_assurance_log` reruns every event mechanism

Mechanism surface: TRUST transfer

Current status: Implemented edge-local proposition-dispatch reference mechanism

Ownership boundary: `record_trust` binds one exact transformation, its complete inputs and output, the historical Graph digest, and the prerequisite TRUST event for every derived input. Recursive current-admissibility checking excludes the route if any required event, artifact, or transformation degrades or conflicts, without deleting history. No universal scalar support algebra exists.

Mechanism surface: ROT derivation and cross-surface propagation

Current status: Implemented bounded reference mechanisms

Ownership boundary: Strictly degrading status propositions and complete challenge-ledger projections produce additive current-state overlays; the deduplicated descendant impact set is discovery, and a descendant status change still needs its own mechanism

Mechanism surface: RUST concentration, localization, and diagnostic attribution

Current status: Implemented bounded reference mechanisms

Ownership boundary: A passing descendant-deviation proposition produces deduplicated reverse reachability; concentration counts unique descendants; localization and BLAME require separate passing propositions. GUILT additionally composes exact responsibility, obligation-applicability, and obligation-violation components; an opaque obligation label cannot establish it.

Mechanism surface: Complete native PASS/FAIL/UNKNOWN/CONFLICTED derivation for arbitrary hidden-witness predicates

Current status: Experimental and unimplemented

Ownership boundary: This is distinct from the implemented bounded composed-graph untraversability analyzer: that analyzer checks reachability of declared protected facts under one finite evidence-backed knowledge-hypergraph contract; it does not prove arbitrary hidden-witness predicates. A caller-supplied state tag is not an earned verdict.

Mechanism surface: Specific optional proof backends

Current status: Backend-specific maturity; the RISC Zero example has one recorded native proof

Ownership boundary: Optional proof machinery cannot make the governing architecture optional or establish broader VSTD conformance

5.5 Serialized receipt dispatch

Dispatch first by `schema_version`, then by the required profile discriminator. VSTD-1 claim receipts require `receipt_kind = "claim_mechanics"`; generic-run receipts require `receipt_kind = "generic_`

`computational_run`". Unknown identifiers, absent discriminators, and mismatched shapes fail closed. The current reader does not infer a profile from retired pre-current-profile field arrangements.

5.6 Installed specification ownership

Every `standard/*.md` file has a byte-identical installed resource under `src/verifier/specifications/`. Verifier descriptors use those resources when no source checkout is present. The installed-wheel gate runs outside the checkout and rejects an unavailable specification digest.

The artifact-control schema has a byte-identical installed copy under `src/verifier/artifact_control/`. GitHub Pages publishes it at its declared `/schemas/` route alongside receipt schemas without reclassifying it as a receipt.

5.7 Generic-run validation contract

`vstd validate` is an integrity/profile validator for the `generic_computational_run` profile. It enforces the strict receipt shape and recomputes the stable-payload digest. The dynamic path keys inside `source_state` `.source_file_hashes`, unconstrained recorded evaluator values, and additional declarations inside `assessment_context.refutation_surface` are explicit data or extension surfaces. The refutation surface remains open for caller-defined domain refutations, but every additional value remains a declaration until an applicable mechanism checks it. Unknown object properties outside those named surfaces fail closed.

Validation does not rehash referenced artifacts, rerun the command, resolve evidence references, or establish that recorded declarations are true. Those are separate mechanisms. `validate`, `inspect`, and `reproduce` honor `--json` for generic-run and VSTD-Graph receipts; the envelope reports command completion without upgrading the receipt's claim semantics.

5.7.1 Generic-run assessment context

`assessment_context` is a required VSTD-1 generic-run container, not a VSTD-4 object. It participates in the canonical digest and retains the manifest-declared mechanism, resource-bound, commitment, and refutation coordinates without using a numbered-profile identifier as a generic container name.

Member: verifier identity

Five-As role: Assessment

Maximum current meaning: Names the generic mechanism; identity alone earns no result.

Member: specification identity

Five-As role: Attribution and Assessment

Maximum current meaning: Binds the mechanism to VSTD-1 bytes, not VSTD-4.

Member: implementation identity

Five-As role: Assignment

Maximum current meaning: Identifies implementation bytes; it does not prove an independent implementation.

Member: parser identity

Five-As role: Assignment

Maximum current meaning: Identifies parser bytes; equality with the implementation hash records shared bytes, not separation.

Member: format identity

Five-As role: Attribution

Maximum current meaning: Names the generic capture/validate/reproduce fragment.

Member: resource bounds

Five-As role: Assurance input and Assessment bound

Maximum current meaning: Records manifest declarations; the generic runtime does not establish their enforcement.

Member: prior commitment

Five-As role: Assurance input

Maximum current meaning: Records a commitment string; receipt inclusion does not prove temporal priority.

Member: refutation surface

Five-As role: Attribution

Maximum current meaning: Declares admissible refutations and exclusions; it is not the checked VSTD-4 RefutationSurface.

The open refutation surface may carry a `platform_comparability` declaration naming the compared subject mechanism, compatible operating systems, and result surfaces. The declaration remains data until `compare_platform_run_receipts` or `vstd compare-platforms` validates one canonically intact receipt per declared system, verifies equal non-platform bindings, and compares the declared result projections. Missing or duplicate platforms and binding drift remain `NOT_ESTABLISHED`; malformed or contradictory receipts are `INVALID`; only result disagreement after comparability is established is `CONFLICTED`. `PASS` establishes agreement only for the supplied recorded results. Python serializes macOS as `Darwin`. The comparator also binds normalized machine family so the current hosted Linux, Windows, and Intel macOS observation isolates the operating-system dimension more narrowly. It does not attest native execution, virtual-machine identity, semantic correctness, universal portability, or actor independence, and it does not create a new receipt schema.

Closure coordinates identify assessment questions; they are not containers for generic verification context. The neutral container must not generate profile-numbered binding structures. Nothing in `assessment_context` supplies a VSTD-4 result.

5.8 Five-As human traversal

Status: non-serialized architecture guide. The five As are roles in a human traversal of existing VSTD records, not five new object types, numbered profiles, statuses, or a scalar assurance score. No receipt or schema format is defined here.

`ASSURANCE_0 -> ATTRIBUTION -> ASSIGNMENT -> ASSESSMENT -> ASSURANCE_1` reads as follows:

Stage: `ASSURANCE_0`

Operational meaning: Identified evidence or a previously assessed claim, with its evidence basis, provenance, bounds, trust roots, limitations, current state, and unresolved conflicts or unknowns.

Existing VSTD machinery: Generic-run receipts and external-evaluation evidence; `EvidenceClassification`; Graph artifacts, statuses, and `ConflictRecord`; VSTD-3 evidence sources, gaps, and claim evaluations; VSTD-4 certificates and kernel results.

Current gap: There is no universal Assurance record or cross-profile scalar ordering.

Stage: ATTRIBUTION

Operational meaning: The explicit relation from evidence to the exact subject/predicate it supports, including the mapping, extraction, or transformation, scope, bounds, provenance, and information loss.

Existing VSTD machinery: Generic-run bound-output extraction and recorded external references; Graph transformation hyperedges; VSTD-4 `ClaimCoordinate`, `ClaimBinding`, and `Grounding`; loss-sensitive SCITT coordinates.

Current gap: Mapping and loss declarations remain profile-specific; a reference alone is not a checked mapping.

Stage: ASSIGNMENT

Operational meaning: The most precise evidenced execution coordinate available: computation, execution instance, software/runtime, machine/substrate, then optional actor/operator bindings. Missing coordinates remain partial or UNKNOWN.

Existing VSTD machinery: Generic-run execution and source-state records; VSTD-3 `WorkloadIdentity`, `ExecutionIdentity`, topology, device, runtime, and evidence-source records; VSTD-1 `independence_basis` for the separate independence question; generic `BoundProposition` mechanism dispatch for an exact assignment proposition.

Current gap: The legacy generic-run declaration does not self-promote into an evidenced Assignment. A deployment supplies the mechanism and observations; Assignment alone establishes no trust, authorization, independence, or responsibility.

Stage: ASSESSMENT

Operational meaning: An identified verifier or mechanism evaluates one bounded proposition under the applicable input Assurance, Attribution, Assignment, specification/profile, trust roots, and bounds. It earns only the predicates it checks.

Existing VSTD machinery: Generic validation, artifact rehash, and rerun mechanisms; VSTD-3 recomputed `ClaimEvaluation`; Graph validation and candidate-profile certificates; the VSTD-4 grounded certificate kernel; native VSTD plus native SCITT composition.

Current gap: No one verifier covers every profile; mechanism results remain adjacent rather than silently merged.

Stage: ASSURANCE_1

Operational meaning: The assessment output recorded as new evidence with complete lineage to its inputs, mechanism, proposition, and limits. It may be `PASS`, `FAIL`, `UNKNOWN`, `CONFLICTED`, or a profile-specific equivalent.

Existing VSTD machinery: Receipts, claim evaluations, kernel results, certificates, artifact digests, Graph artifacts/hyperedges, prior commitments, and the replayable `verifier-graph-assurance-1` event envelope preserve and reference the output.

Current gap: The Graph envelope is not a universal scalar or an automatic cross-profile cast; each later loop still names and reruns its mechanism.

First-hand and second-hand describe **provenance**, not strength. A first-hand self-observation may be weak; a second-hand certificate may be strongly bound to a narrow proposition. `EvidenceClassification` records how evidence entered a profile, but its name, source, or placement never substitutes for the profile's verification mechanism.

The smallest operational loop is:

1. select one proposition and retain every applicable input state, limitation, conflict, unknown, trust root, and freshness bound;
2. bind each input to that proposition through an inspectable attribution, preserving transformations and declared information loss;
3. record Assignment only to the specificity evidenced, leaving absent coordinates UNKNOWN;
4. run the named assessment mechanism under its specification and bounds; and
5. record the output as a new evidence artifact and transformation, without changing any input record. A later loop may consume that output only as lineage-preserving input to another explicitly identified assessment.

No semantic strength is gained by storage location, field name, repetition, graph multiplicity, actor reputation, or propagation. Every increase in assurance must identify the verification mechanism that earned it.

This is the human forward traversal of the same topology VSTD-Graph stores for machines. TRUST is bounded, mechanism-earned support across an admissible recorded transformation; the child still discharges its new obligations. ROT is typed, time-indexed degradation of current admissibility while historical evidence remains immutable. RUST is inverse-TRUST diagnostic reachability from a downstream deviation toward recorded ancestors. Together they describe memetic causal-provenance and lifecycle behavior over one development graph. They do not establish actor standing, moral character, responsibility, or automatic ancestor falsification. The reference runtime emits bounded TRUST, ROT, and RUST events only after their exact mechanisms run. Causal localization and artifact-relative BLAME or GUILT require additional exact propositions. BLAME establishes bounded responsibility or material contribution. GUILT is not directionally opposite: it composes three separately bound passing evaluations for responsibility, applicability of an exact scoped obligation, and violation of that same obligation relative to the same localized deviation. The final evaluation binds all three component digests. One compound mechanism may emit those three evaluations in one invocation, but one opaque combined result cannot substitute for them. Neither becomes actor reputation, moral character, automatic legal liability, or social scoring; missing GUILT does not establish innocence or exoneration.

5.8.1 Recursive-amplification falsification outcomes

Probe: Duplicate evidence or a duplicate identifier

Required outcome: No extra support; public receipts reject duplicates and in-memory graph construction rejects replacement.

Probe: Duplicate graph paths

Required outcome: Reachability is set-valued; path count never raises assurance or the candidate Graph profile.

Probe: Repeated identical reruns

Required outcome: At most the same bounded equivalence result; repetition does not prove independence or a stronger reproduction state.

Probe: Assessment consumes its own output

Required outcome: Invalid within that assessment; an output can enter only a later, distinct assessment with preserved lineage.

Probe: $A \rightarrow B \rightarrow A$ or a self-loop

Required outcome: Invalid Graph topology; acyclicity checking rejects the loop.

Probe: Second-hand evidence relabeled first-hand

Required outcome: Provenance conflict or unsupported declaration; no strength change.

Probe: Attribution without a checked mapping

Required outcome: Declaration or UNKNOWN, never mapped support.

Probe: Machine Assignment treated as responsibility

Required outcome: Prohibited inference; Assignment records execution coordinates only.

Probe: Actor identity treated as trust

Required outcome: Prohibited inference; identity and reputation do not strengthen an artifact result.

Probe: Conflicted upstream evidence collapsed

Required outcome: Conflict remains explicit and blocks a clean candidate Graph profile.

Probe: Stale, revoked, challenged, or unknown evidence reused as clean current support

Required outcome: Inadmissible to a clean current Graph candidate; this is ROT in current admissibility, not revision of the historical record.

Probe: Recursive propagation with no new mechanism

Required outcome: No transition from ASSURANCE_0 to stronger ASSURANCE_1; lineage growth is not assurance growth.

5.9 Separation and Graph boundaries

The historical `independent_audit` field name does not prove independence. Its `independence_basis` records actor, implementation, and runtime separation. Repeated or matching results are artifact agreement, not evidence that separate actors performed the runs; absent separation evidence is NOT_DEMONSTRATED. Serialized

status words and evidence-reference strings cannot self-promote that result. The generic-run compatibility path treats supplied assertions as no stronger than **DECLARED**, rejects receipts that label them **EVIDENCED**, and never derives **EVIDENCED**. The distinct VSTD-5 reference path can establish only its exact declarant/witness separation propositions after all seven seams are rerun by registered mechanisms. Typed binding, identity, separation, and corroboration errors keep **computed_independence** fail-closed; no error-message text is interpreted as a semantic category. Witness identities and independence assertions serialize as separate ordered arrays, so duplicates, orphan assertions, and missing cardinality survive receipt build and replay rather than disappearing inside a keyed map. Permissive assessment remains separate from portable-record admission: the builder raises unless the result inhabits the strict VSTD-5 schema and embeds every verdict-material evidence byte, and the rechecker enforces the same zero-dependency shape gate before replay. It also compares every redundant VSTD-4 entry coordinate and treats **corroboration_class** as part of the mechanism-checked expected proposition. The admitted evidence-bound VSTD-4 result retains its exact **claim_id**, and VSTD-5 requires the bundle to use that identifier; a shared binding or certificate digest cannot turn a neighboring identifier into an alias. Thus relabeling any of these coordinates cannot retain an established result. This path does not upgrade the legacy generic-run fields.

Graph conflict records retain incompatible values and their evidence references without adding a scalar score or changing the frozen artifact-status vocabulary. A conflict makes the subject inadmissible to a clean candidate Graph profile. The frozen **verifier-data-1** reader retains its historical separate artifact/transformation namespaces. Direct new construction, evidence-bound Graph establishment, and the assurance overlay require global cross-kind disjointness, so an untyped **subject_id** cannot ambiguously name both kinds of Graph object.

5.9.1 Recursive current-state audit

Historical receipt bytes and their recorded **PASS** remain unchanged. A later current-state question is a new assessment over the retained graph and applicable lifecycle records:

Scenario: An ancestor is **CHALLENGED**, **REVOKED**, or **STALE**

Implemented outcome: Candidate/evidence-bound Graph recomputation follows the full ancestor closure and returns compatibility field **level = 0**. **AssuranceLedger** also records typed ROT or projects append-only challenge-ledger state into a derived current view; historical Graph bytes remain unchanged.

Scenario: An ancestor is **SUPERSEDED**

Implemented outcome: The historical Graph candidate remains admissible by design; the stricter all-ancestors-**VALID** policy rejects it for current-use admission. Supersession does not retroactively falsify its prior lineage role.

Scenario: Upstream evidence conflicts

Implemented outcome: A retained **ConflictRecord** or a mechanism-established **CONFLICT_DECLARATION** event blocks every dependent edge-local **TRUST** route. **resolve_conflict** accepts only a mechanism-passing proposition bound to the exact conflict and one retained competing value. A status resolution projects the selected artifact or transformation state: only **VALID** / **COMPLETED** can restore the route. An arbitrary resolved predicate remains admissibility-blocking because value adjudication does not establish support effect; no general non-status admissibility-effect mechanism is implemented. The original conflict and resolution evidence remain historical.

Scenario: Evidence arrives by multiple paths or one run receipt repeats a reference

Implemented outcome: Reachability and impact sets deduplicate identifiers. Multiplicity supplies no independence or strength.

Scenario: A descendant deviation points toward shared ancestors

Implemented outcome: A mechanism-passing deviation emits RUST over the deduplicated historically recorded contributing ancestor set. Current revocation or conflict does not erase diagnostic history. Structural concentration counts unique deviating descendants, not paths or causal strength. Localization selects and binds one exact passing RUST event, its descendant-deviation binding digest, and an ancestor contained in that event. BLAME and GUILT bind that localization event and require separate exact mechanisms.

Scenario: A challenge ledger changes a claim's current status

Implemented outcome: `project_challenges` binds its complete append-only records into a current Graph overlay and embeds those records for replay. Existing TRUST remains historical, while recursively dependent events disappear from `current_trust_events`; `impacted_descendants` reports the deduplicated reassessment surface. It never mutates the historical graph.

Scenario: Later evidence adjudicates a conflict

Implemented outcome: The additive resolution retains the original competing evidence and its mechanism evaluation. VSTD-Graph-1 receipts remain immutable; the separate assurance overlay owns the resolution and current-state projection. Removing or rewriting historical evidence remains invalid.

Scenario: Candidate calculation encounters cyclic ancestry

Implemented outcome: Rejected before candidate calculation; recursive topology cannot manufacture assurance.

The forward blast-radius query remains discovery only. **AssuranceLedger** is the distinct binding mechanism for explicit edge-local TRUST, ROT, RUST, challenge projection, and conflict declaration/resolution. `recheck_assurance_log` reconstructs the historical Graph, rehashes every embedded evidence item, reruns the exact registered mechanisms, reproduces the event chain, and compares the derived current view. The ledger never infers an unrecorded edge, converts topology into assurance, or treats RUST as causality. Domain-specific transfer and localization propositions still require their registered mechanisms and may return UNKNOWN.

Chapter 6

Claims, Limits, Boundaries, and Refutation Mechanics

Acronyms: artificial intelligence (AI); Advanced Micro Devices (AMD); application programming interface (API); Amazon Web Services (AWS); Concise Binary Object Representation (CBOR); CBOR Object Signing and Encryption (COSE); conjunctive normal form (CNF); Device Identifier Composition Engine (DICE); floating-point operation (FLOP); grounded decision certificate (GDC); identifier (ID); machine learning (ML); NVIDIA Management Library (NVML); JavaScript Object Notation (JSON); Secure Hash Algorithm 256-bit (SHA-256); Secure Hash Algorithm 3 256-bit (SHA3-256); system management interface (SMI); Security Protocol and Data Model (SPDM); Software Package Data Exchange (SPDX); Supply Chain Integrity, Transparency, and Trust (SCITT); trusted computing base (TCB); Coordinated Universal Time (UTC); Verifier Standard (VSTD).

Reader aid: **concept glossary and primary precedents.**

Status: normative interpretation guide for the VSTD object and Graph ladders

This guide translates VSTD claim language into ordinary language. When a short claim conflicts with the bounded wording here, the bounded wording controls.

6.1 Reading every VSTD claim

A VSTD result always has this form:

For this **identified subject snapshot**, this **identified mechanism** returned this **bounded result** over this **declared verification surface**, using this **bound evidence**, subject to these **limitations, trust roots, and horizons**.

Omitting any bolded idea changes the claim. **VERIFIED** never means universally true, safe, complete, permanent, legally authorized, or endorsed.

6.2 Skeptical review summary

VSTD has no single scalar “strongest claim”; mechanisms establish different predicates. The strongest generally reusable implemented statement is therefore an exact, bounded checker result—not a claim of universal truth or whole-project conformance.

Reviewer question: What can the generic validator establish?

Current answer: Stable receipt content and strict profile shape.

Mechanism and trust roots: Canonicalization, recorded digest, profile discriminator, and bundled validator bytes.

Boundary or missing mechanism: It does not verify the recorded native claim, external evidence, actor identity, or independence.

Reviewer question: What can the grounded-certificate kernel establish?

Current answer: The exact VSTD4-GDC-1 decision was accepted, rejected, or left UNKNOWN under its claim binding and resource bound.

Mechanism and trust roots: Certificate bytes, formula, grounding, policy/evidence roots, verifier descriptor, and kernel.

Boundary or missing mechanism: Kernel acceptance alone is not VSTD-4 conformance. The evidence-bound path separately reruns every prerequisite/rung mechanism before it may establish conformance.

Reviewer question: What can VSTD-Graph establish?

Current answer: Stored topology plus a candidate over supplied ratings, or an evidence-bound Graph profile after every complete-closure rating mechanism is rerun.

Mechanism and trust roots: Graph bytes, lifecycle/conflict view, exact rating bindings, embedded evidence, mechanisms, roots, bounds, and certificate.

Boundary or missing mechanism: Recorded topology is not complete real-world causality; the compatibility path remains NOT_ESTABLISHED, and domain mechanism correctness remains a declared trust boundary.

Reviewer question: What can VSTD-3 establish?

Current answer: Conditional device, firmware, execution, accounting, continuity, or fleet predicates when each required evidence path validates.

Mechanism and trust roots: Named roots, keys, nonces, measurements, topology, events, appraisal inputs, and profile-specific validators.

Boundary or missing mechanism: Host inventory is not attestation; production vendor integration and complete mediation outside the emulator remain separate requirements.

Reviewer question: What can artifact control establish?

Current answer: Current exact file bytes and paths match a freeze manifest; an optional finite seal closes that freeze; with an actual supplied and cleanly verified parent whose recorded coordinates agree, a thawed descendant currently matches or differs from that parent.

Mechanism and trust roots: Preserved bytes, SHA-256 and SHA3-256 commitments, read-only payload-tree guard, Ed25519 signature, artifact-derived identifiers, supplied parent bundle, any supplied external artifact/key anchor, fail-closed final-entry classification for supported creation paths, and ordinary lexical-type checks for authoritative internal bundle members.

Boundary or missing mechanism: Read-only is not privileged access control; a seal is not encryption, correctness, trusted time, ownership, durable external archiving, or a numbered-profile result. A thaw sidecar alone does not authenticate a parent or historical copy operation. Outer read aliases remain distinct from internal closure, ordinary hard links do not prove exclusive inode ownership, and path checks do not establish universal race-free, mount-independent, or network-filesystem security.

Reviewer question: What can interoperability loading, analysis, and experimental planning establish?

Current answer: Strict loading can establish that supplied VSTD-2 JSON has the supported structure and valid internal references; analysis identifies holes already represented by the resulting typed geometry; experimental planning identifies exact declared candidates in one bound component registry.

Mechanism and trust roots: Supplied JSON, strict loader, typed geometry and digest, registry digest, exact schema, interaction-mode, relation, and mechanism coordinates, component descriptors, prerequisites, blockers, and the nonexecuting planner.

Boundary or missing mechanism: Successful loading does not establish the truth or completeness of the declared surface. A candidate is not checker execution, evidence, a native result, validation, closure, safety, authority to act, critical-domain readiness, or VSTD conformance.

Reviewer question: What does SCITT add?

Current answer: Signature and registration/inclusion evidence for exact payload bytes under a declared relying-party policy.

Mechanism and trust roots: Native SCITT verifier, issuer/log keys, payload digest, registration policy, and Transparency Service evidence.

Boundary or missing mechanism: Registration cannot establish payload correctness, VSTD conformance, or issuer authority outside the policy. The current example uses a local test log.

Reviewer question: What remains outside current support?

Current answer: General AI safety, hidden state, complete physical-world history, automatic real-world actor independence, unrecorded provenance, universal support algebra, and unqualified truth.

Mechanism and trust roots: VSTD-5 and Graph assurance now dispatch exact evidence-bound mechanisms; they do not manufacture the missing domain observations or external witnesses.

Boundary or missing mechanism: Preserve UNKNOWN, UNSUPPORTED, CONFLICTED, or NOT_ESTABLISHED; do not infer a clean result.

In the experimental interoperability path, **validation** names the future process of attempting to discharge verification-surface holes with authorized component execution and bound evidence, followed by reanalysis. Detection and planning alone are not that process.

The **artifact-awareness contract** applies alongside these limits: permission does not establish knowledge, and withholding direct access does not establish non-inferability. Actual awareness, permitted awareness and potentially inferable awareness remain separate. Version 1.3.0 includes an experimental finite composed-graph analyzer: **MATCH** means only that complete bounded closure of the declared evidence-backed model did not reach a protected fact at the exact observer, mode, interval, query, capability and resource coordinate. It is not an awareness tracker, open-world non-inferability proof, or enforcement of confidentiality of awareness.

Graph-topology interpretation can retain paradox candidates, simultaneous constraint loops and explicit backward-time relations. Its bounded Boolean and temporal consistency results concern only the exact encoded interpretation, not physical causation, uniqueness, complete modeling, receipt validity or conformance. Structural cycles do not prove contradictions; circular references cannot manufacture mechanism-earned forward artifact support. Missing semantics remain NOT_ESTABLISHED.

Every claim below expands one of these boundaries into publishable wording and its required falsification surface.

6.3 Claim translation table

Claim: “This receipt’s stable content has not changed.”

May it be made?: Yes, after validation passes.

Why: The validator recomputes the canonical digest over the specified stable fields and compares it with the recorded digest.

Required evidence: Receipt bytes, canonicalization version, recorded digest, passing validator result.

What it does not mean: The statements inside the receipt are true or authentic.

Claim: “These observed bytes match this SHA-256 digest.”

May it be made?: Yes, conditionally.

Why: A named mechanism can hash accessible bytes at an observation time and compare them with the recorded digest.

Required evidence: The bytes, hashing mechanism, observation time, expected digest, comparison result.

What it does not mean: The bytes came from the claimed source, existed before observation, are uncontaminated, or are legally usable.

Claim: “This artifact is frozen.”

May it be made?: Yes, after freeze verification passes.

Why: The supplied source was an accepted ordinary file or directory; the current regular-file bytes, portable paths, manifest identifiers, and read-only payload-tree guard recompute.

Required evidence: Complete bundle, passing `vstd artifact verify -freeze-only`, and the exact mechanism version.

What it does not mean: A symbolic-link source was frozen as its target, privileged mutation is impossible, an external archive retained the artifact, the artifact is correct, or a signature exists.

Claim: “This artifact is sealed.”

May it be made?: Yes, after seal verification passes.

Why: The carried Ed25519 key verifies the finite signature closure and the seal identifier closes the signature-bearing envelope.

Required evidence: Passing seal verification plus an expected artifact/key coordinate when whole-bundle substitution is in scope.

What it does not mean: Encryption, secrecy, ownership, authorization, trusted time, semantic correctness, continuous custody, actor trust, or a numbered VSTD profile result.

Claim: “This thawed descendant currently matches this sealed parent.”

May it be made?: Yes, only when that actual parent is supplied and verifies.

Why: The parent must be cleanly `SEALED`; every sidecar parent coordinate and recorded seal must agree; and the descendant identity is recomputed from authoritative parent kind and media type.

Required evidence: Descendant, strict thaw sidecar, supplied parent bundle, passing parent verification, exact coordinate comparison, and external artifact/key anchor when continuity is required.

What it does not mean: The sidecar proves its own history, `thaw_artifact` was independently observed, or the supplied parent has external continuity when no external coordinate was checked. Sidecar-only agreement remains `NOT_ESTABLISHED`.

Claim: “VSTD-Graph records this lineage graph.”

May it be made?: Yes.

Why: The receipt binds the stored artifact nodes, transformation edges, roles, statuses, and declarations. Historical Graph-1 receipts retain the serialized receipt identifier `verifier-data-1`.

Required evidence: Valid receipt and structurally valid hypergraph.

What it does not mean: The graph contains every real-world input or transformation.

Claim: “This is the complete provenance of the model or dataset.”

May it be made?: No, unless completeness is independently evidenced for the declared boundary.

Why: A graph cannot infer hidden inputs, pre-observation history, out-of-band processing, or missing instrumentation.

Required evidence: Independent coverage evidence for every declared boundary plus explicit horizons outside it.

What it does not mean: That a high coverage summary proves complete real-world lineage.

Claim: “This transformation actually ran and produced this output.”

May it be made?: Only with execution evidence.

Why: Software, parameters, and environment fields are declarations until a run trace, rerun, attestation, or equivalent evidence binds execution to the output.

Required evidence: Identified inputs and outputs, execution trace or rerun, software identity, parameters, environment, and evidence classification.

What it does not mean: Recording a script name or commit proves execution.

Claim: “The recorded Boolean provenance policy passed.”

May it be made?: Yes, when the policy result validates.

Why: The reference solver evaluates the recorded CNF formula.

Required evidence: Formula, variable map, graph snapshot, solver identity, passing result.

What it does not mean: The prose-to-formula translation was complete, the external facts were true, or broader policy compliance was established.

Claim: “No recorded ancestor is marked `REVOKED`.”

May it be made?: Yes, if that narrow query passes.

Why: The query checks exactly the recorded `REVOKED` status over the ancestor closure.

Required evidence: Target artifact, recorded graph, status values, passing policy result.

What it does not mean: Every ancestor is **VALID**; **UNKNOWN**, **CHALLENGED**, or **STALE** may still exist unless explicitly rejected.

Claim: “All recorded target ancestors are explicitly **VALID**.”

May it be made?: Yes, if the fail-closed valid-ancestor policy passes.

Why: That policy rejects every recorded target ancestor not explicitly marked **VALID**.

Required evidence: Target artifact, ancestor closure, status evidence, passing **POL-ALL-ANCESTORS-VALID**.

What it does not mean: The status declarations are authentic or that unrecorded ancestors do not exist.

Claim: “The recorded **SPDX** metadata matches the allowlist.”

May it be made?: Yes, if the exact metadata policy passes.

Why: The policy compares recorded license identifiers with the declared allowlist.

Required evidence: Rights records, roots, allowlist, passing policy result.

What it does not mean: Copyright ownership, license authenticity, compatibility, fair use, or a legal ruling.

Claim: “This result reproduced bitwise.”

May it be made?: Yes, for the declared outputs after a passing rerun.

Why: The rerun produced byte-identical declared output artifacts.

Required evidence: Original receipt, runnable command, captured inputs, environment boundary, rerun outputs, byte comparison.

What it does not mean: All environments will reproduce it or the computation is empirically correct.

Claim: “These Linux, Windows, and macOS results are cross-platform comparable.”

May it be made?: Yes, only for the supplied declared surfaces after the comparator returns **PASS**.

Why: Every declared platform supplied one canonically intact generic-run receipt; non-platform claim, source, command, Python, normalized machine-family, and mechanism bindings matched; and the declared recorded result projections matched. Python records macOS as **Darwin**.

Required evidence: All declared receipts, their canonical digests, the shared **platform_comparability** declaration, comparison-binding digest, result digests, exact comparator implementation, and hosted-runner evidence when native operating-system execution is claimed.

What it does not mean: The declaration proved compatibility, all machines or versions behave identically, the result is semantically correct, the receipt itself attests native execution, or independent actors performed the runs. Missing or non-comparable evidence remains **NOT_ESTABLISHED**; comparable disagreement is **CONFLICTED**.

Claim: “This was independently verified.”

May it be made?: Only when distinct producer and checker actors plus the relevant execution seams are evidenced.

Why: Matching results establish artifact agreement, not who performed either run. Actor independence, implementation separation, runtime separation, and the trusted computing base must be recorded separately.

Required evidence: Evidence binding distinct actors to the producer and checker runs, implementation/runtime isolation, trusted computing base, and the checker result.

What it does not mean: Two runs, two processes, two machines, or matching outputs automatically prove independent actors.

Claim: “This verification surface is self-closed.”

May it be made?: Only if every VSTD-2 self-closure condition passes.

Why: Self-closure requires ordinary closure, resolved material residuals, discharged valences, post-verified mechanisms, no unresolved trust-root horizon, and contiguous verification orders.

Required evidence: Complete geometry document and passing closure assessment with no blockers.

What it does not mean: Universal truth, infinite regress closure, permanent validity, or verification outside the surface.

Claim: “This competition submission and score are bound together.”

May it be made?: Yes, conditionally.

Why: A receipt can bind identified submission bytes, evaluator version, raw metrics, and deterministic score derivation.

Required evidence: Submission digest, evaluator/scorer identity, environment, raw metrics, score rule, receipt.

What it does not mean: Hidden-test integrity, no leakage, leaderboard ranking, prize eligibility, or organizer acceptance.

Claim: “This registered component is an exact candidate for this modeled surface hole.”

May it be made?: Yes, only after exact catalog matching in a registry-bound plan.

Why: The component’s declared schema and interaction mode plus the hole’s relation and mechanism coordinates, where present, matched exactly; domain tags did not participate.

Required evidence: Structurally valid typed geometry, geometry digest, exact hole, complete component descriptor, registry version and digest, planner implementation, candidate status, prerequisites, and blockers.

What it does not mean: The component ran, was available at execution time, produced evidence, discharged the hole, was independently checked, established closure or safety, or is ready for a critical domain.

Claim: “This native verifier result was mapped into VSTD.”

May it be made?: Yes, when the mapping preserves the native object, result, trust roots, bounds, and unsupported fields.

Why: VSTD can standardize the claim boundary and portable result semantics around a domain verifier without performing that verifier’s native work.

Required evidence: Native object and version, native verifier implementation/version, native result, per-field mapping, information-loss declaration, VSTD coordinate, adapter tests.

What it does not mean: VSTD replaced or reimplemented the native verifier, strengthened its result, inherited its authority, or established conformance to the source standard.

Claim: “A challenge to this recorded ancestor affects these recorded descendants.”

May it be made?: Yes, as a bounded reassessment surface.

Why: `project_challenges` reruns the built-in projection over complete challenge records; `impacted_descendants` deduplicates forward reachability; current TRUST records depending on the now-inadmissible ancestor are excluded.

Required evidence: Challenged artifact ID, bound hypergraph, complete challenge records, and replayed assurance log.

What it does not mean: Historical receipts or TRUST events were mutated, every descendant is false, or unrecorded downstream systems were found.

Claim: “This artifact has bounded technical GUILT for this deviation.”

May it be made?: Only after component composition passes.

Why: The reference ledger requires separately bound passing responsibility, exact obligation-applicability, and same-obligation violation evaluations whose artifact, deviation, localization, and scope coordinates agree; the final mechanism binds their exact digests.

Required evidence: Exact artifact and descendant IDs, passing localization and RUST lineage, typed obligation coordinate and scope, all three component events and evidence, mechanisms and implementation digests, trust roots, bounds, final composition, and successful replay.

What it does not mean: Moral character, actor reputation, social scoring, automatic legal liability, innocence or exoneration when absent, obligation satisfaction, or absence of hidden contributors.

Claim: “The compatibility API returned Graph `level = N`.”

May it be made?: Not yet as a conformance claim.

Why: The current implementation computes candidate Graph profile `N` from caller-supplied artifact and edge ratings; `level` is the retained field name for that profile number. It labels the result `CALLER_SUPPLIED` and `NOT_ESTABLISHED`.

Required evidence: A structurally valid graph and explicit supplied ratings. Conformance additionally requires implemented rating-to-evidence bindings for every required profile coordinate.

What it does not mean: The supplied ratings were independently derived, every coordinate’s evidence passed, or Graph conformance was established.

Claim: “The evidence-bound Graph path established profile `N`.”

May it be made?: Only for profile 1–5 at the exact collection and current view under the rerun mechanisms.

Why: Every member, ancestor, and reached edge rating binding passed; each rating binds the Graph bytes, deduplicated member set, collection, and claim; the Graph certificate checked; embedded evidence permits replay.

Required evidence: Exact Graph/event-log bytes, bindings, evidence, mechanism digests, trust roots, bounds, and recheck result.

What it does not mean: Profile zero is conformance, real-world lineage is complete, topology proves causality, or another collection inherits the result.

6.4 Competition and scored-evaluation claims

For predictive-AI, scientific-ML, agent, and other scored evaluations, bind the exact rules, data, model, submission, evaluator, metrics, score, transformations, environment, and evidence classes. Mark hidden tests as a horizon—not evidence of integrity. This adds no verdict, affiliation, certification, endorsement, ranking, prize eligibility, or organizer acceptance.

For later-resolved predictions, also bind emission and resolution times, the frozen prediction digest, update or abstention policy, resolution source and digest, scoring rule, and channel independence. Corrections are additive; never overwrite a frozen prediction. See the complete non-normative **competition profile**.

Use the coordinate-bounded wording:

The submission and score receipt binds the declared artifact, evaluator, and provenance surface.
Hidden-test integrity and organizer acceptance remain outside the participant-observable surface.

Do not shorten this to “the model,” “the competition result,” or “the prediction is verified.”

6.5 VSTD-4 grounded-decision claim translations

VSTD4-GDC-1 makes a decision certificate checkable outside its producer against an explicit claim coordinate, formula, grounding map, verifier identity, resource bounds, and prior commitment. It does not make the certificate independent of the evidence source or make the grounded claim true outside that coordinate.

Claim: “This VSTD4-GDC-1 certificate was accepted.”

Bounded translation: The identified reference kernel accepted the exact canonical certificate under the declared claim binding, fragment, verifier, and resource bounds.

Required qualification: Name the certificate digest, implementation commit, claim coordinate, cost tier, bounds, and kernel result.

Prohibited stronger inference: The underlying evidence is authentic, the policy captured every intended condition, or the claim is globally true.

Claim: “This decision is grounded.”

Bounded translation: Every variable and clause in the accepted certificate maps to declared subjects, predicates, values, and encoding rules whose roots are bound by the certificate.

Required qualification: Preserve the evidence root, policy root, grounding map, and exclusions.

Prohibited stronger inference: Unrecorded evidence does not exist, the grounding source is independent, or the physical world is completely represented.

Claim: “The reference implementation computed VSTD-4 candidate depth k .”

Bounded translation: Caller-supplied nonempty references were structurally consistent through rungs $1..k$ and, when $k < 14$, the candidate ceiling certificate blocks rung $k+1$.

Required qualification: State `CANDIDATE`, `conformance_status = NOT_ESTABLISHED`, the supplied references, certificates, budgets, and horizons.

Prohibited stronger inference: The references establish their rung propositions, VSTD-1/2/3 passed, normative VSTD-4 conformance was established, or VSTD-5 entry is permitted.

Claim: “The evidence-bound path established VSTD-4 normative depth 14.”

Bounded translation: Exact VSTD-1/2/3 and fourteen-rung propositions passed after evidence rehash, mechanism selection/execution, bound enforcement, and kernel checking.

Required qualification: Name the receipt, evidence and mechanism digests, trust roots, bounds, implementation coordinate, and recheck result.

Prohibited stronger inference: The mechanisms are universally correct, an outside witness participated, or the claim is true beyond its exact bindings.

Claim: “The result is refutable.”

Bounded translation: The published result exposes a machine-checkable falsification surface and admissible counterevidence within the declared boundary.

Required qualification: Name that surface, the admissible counterevidence, exclusions, and decision rule.

Prohibited stronger inference: A separate party actually attempted refutation or independently witnessed the evidence.

Claim: “The verifier returned UNKNOWN.”

Bounded translation: The declared check could not establish PASS or FAIL within the implemented fragment, available evidence, or resource bound.

Required qualification: Preserve the indeterminacy reason and transcript.

Prohibited stronger inference: The proposition is false, no proof exists, or a larger bound could not decide it.

Claim: “The artifact is ready for VSTD-5 evaluation.”

Bounded translation: Only after the evidence-bound VSTD-4 path establishes VSTD-4 normative depth 14.

Required qualification: `require_vstd5_entry` rejects the compatibility candidate and admits only the distinct established result type.

Prohibited stronger inference: Candidate depth 14, a PASS over the candidate formula, or nonempty references satisfy the gate.

The public reference implementation and its tests are one implementation. This source coordinate does not claim an external implementation, interoperability result, security audit, independent witness, or third-party certification.

6.6 VSTD-3 accelerator claim translations

Every PASS below is conditional on full receipt validation with the required trust keys and appraisal inputs. A recorded `verification_state`: VERIFIED is not enough by itself.

Claim: Positive execution evidence — “Execution E was observed by S.”

May it be made?: Yes, when S actually emitted execution-specific evidence.

Plain-language translation: The named source recorded the identified execution inside its declared observation boundary.

Evidence and verification: Execution identity; source-specific positive observation; authenticated start/observation/end when claiming device evidence; topology binding.

FAIL / UNKNOWN behavior: FAIL for a contradictory binding; UNKNOWN when there is inventory telemetry but no execution-specific evidence.

Prohibited stronger inference: The device originated the evidence, the workload was correct, or all execution was logged.

Claim: Device identity — “Device D presented an authenticated identity.”

May it be made?: Yes, conditionally.

Plain-language translation: A verifier authenticated an identity and bound it to this receipt’s physical/logical subject.

Evidence and verification: Nonce-bound signed evidence, certificate/endorsement chain, trust root, subject/certificate match, freshness.

FAIL / UNKNOWN behavior: FAIL for invalid signature, wrong subject/certificate, or nonce mismatch; UNKNOWN without keys, roots, or an implemented verifier.

Prohibited stronger inference: Firmware was approved, workload ran, counters are exact, or the device mediated every path.

Claim: Firmware integrity — “Device D presented approved firmware measurements.”

May it be made?: Yes, conditionally.

Plain-language translation: Signed measurements matched the verifier’s named reference values and policy.

Evidence and verification: Authenticated device evidence, component measurements, authentic reference values, appraisal result, nonce and freshness checks.

FAIL / UNKNOWN behavior: FAIL for a proven mismatch/invalid signature/staleness; UNKNOWN when reference values, policy, roots, or parser are absent.

Prohibited stronger inference: Every job was logged, firmware has no vulnerabilities, or a reported version alone proves approval.

Claim: Execution attestation — “Execution E has authenticated device/firmware evidence.”

May it be made?: Yes, only with execution-bound attestation.

Plain-language translation: Authenticated evidence binds the execution/workload commitment, device/logical identity, topology, and event record.

Evidence and verification: Valid device/firmware signature and challenge; workload commitments; bound start/observation/end events.

FAIL / UNKNOWN behavior: FAIL for altered or inconsistent bindings; UNKNOWN for host telemetry or opaque vendor evidence not verified by the core.

Prohibited stronger inference: The accounting quantity is exact or complete mediation holds.

Claim: Execution accounting — “Execution E has quantity Q under method M and scope B.”

May it be made?: Yes, with the qualifiers intact.

Plain-language translation: The receipt binds a typed quantity to an execution, source, method, unit, exactness label, and device scope.

Evidence and verification: Bound accounting observation and event; source; method semantics; partition/topology scope; uncertainty for estimates.

FAIL / UNKNOWN behavior: FAIL when an estimate is labeled exact, scope escapes the execution, or payload binding changes; UNKNOWN when counter semantics/evidence are unavailable.

Prohibited stronger inference: Q is exact physical FLOPs outside B, or different counter methods are interchangeable.

Claim: Accounting continuity — “The authenticated sequence is continuous over interval I.”

May it be made?: Yes, conditionally.

Plain-language translation: Every expected sequence position between the evidenced endpoints connects by authenticated rolling roots and valid reset/anchor semantics.

Evidence and verification: Event IDs, epoch/sequence, predecessor/rolling roots, signatures, reset records, and external anchors when claimed.

FAIL / UNKNOWN behavior: FAIL for forks, rollback, wrong roots, invalid signatures, or tampering; UNKNOWN for gaps, unavailable keys, or honestly unanchored history.

Prohibited stronger inference: Events before/after I exist, timestamps are accurate UTC, or all compute paths entered the sequence.

Claim: Complete mediation — “All governed execution paths on D in epoch E were mediated.”

May it be made?: Only for a mechanism that can substantiate path control. The reference emulator can make this claim only inside its API boundary. Commodity adapters currently cannot.

Plain-language translation: Within the exact named boundary, the measured mechanism gates every governed submission path and emits continuous authenticated accounting.

Evidence and verification: Explicit COMPLETE_MEDIATION_ATTESTED capability from verified device/firmware (or test emulator) evidence, passing continuity, complete execution start/end records, and a documented no-bypass boundary.

FAIL / UNKNOWN behavior: FAIL for an evidenced bypass or incomplete recorded execution; UNKNOWN when path control, keys, continuity, or firmware behavior cannot be established; UNSUPPORTED when the mechanism declares no such capability.

Prohibited stronger inference: No ungoverned path exists outside the boundary, no other device computed, or no undeclared compute occurred anywhere.

Claim: Fleet completeness — “All enrolled devices in F reported during T.”

May it be made?: Yes, relative to the exact enrolled boundary.

Plain-language translation: The expected enrolled-member set and observed set matched for the interval, with no missing or unexpected enrolled member.

Evidence and verification: Fleet manifest/boundary, enrollment state, observation, identities, topology, evidence sources, and separately evidenced enrollment completeness.

FAIL / UNKNOWN behavior: FAIL for missing/unexpected members or inconsistent set accounting; UNKNOWN when enrollment completeness or required evidence is absent.

Prohibited stronger inference: The manifest lists every physically present accelerator or every accelerator controlled by an organization.

Claim: Physical-world/global completeness — “No undeclared compute occurred anywhere.”

May it be made?: No. It is mechanically **UNSUPPORTED**.

Plain-language translation: Ordinary receipts have no observation boundary capable of enumerating all physical execution worldwide.

Evidence and verification: No implemented ordinary VSTD evidence set is sufficient.

FAIL / UNKNOWN behavior: This is not converted to **FAIL**; it remains **UNSUPPORTED** because the quantification cannot be observed.

Prohibited stronger inference: Any wording that turns fleet, host, provider, or device receipts into global absence.

6.6.1 What the current adapters can say

- Generic, NVIDIA `nvidia-smi`/NVML-style, AMD SMI, and Intel generic fixtures can say that the collector observed the recorded host-visible metadata and preserved the raw bytes/digest. That is not device attestation.
- Opaque NVIDIA SPDM/NVTrust/NVSwitch or AMD DICE-labeled bytes can be retained without being promoted. Until certificate, nonce, signature, and measurement appraisal run, the result includes `UNVERIFIED_VENDOR_ATTESTATION`.
- A verified Google/AWS/Microsoft provider fixture can say the provider control-plane statement’s test signature matched. It cannot say the tenant accessed or verified physical firmware unless separately verified hardware evidence is referenced.
- `VirtualVSTDAccelerator` can demonstrate nonce binding, measured reference firmware, execution/accounting events, continuity, reset, anchoring, partition lineage, and complete mediation inside the emulator API. It cannot make a physical hardware, tamper-resistance, or commodity-product claim.

6.7 Implemented mechanism versus specification requirement

VSTD documents contain both requirements and implemented reference mechanisms.

- **Implemented claim:** the public Python path and its tests perform the named bounded operation.
- **Conformance requirement:** another implementation must satisfy the stated rule, but the existence of the rule is not evidence that an integration satisfied it.
- **Future or experimental surface:** vocabulary or geometry exists, but broader interoperability or independent implementation evidence is incomplete.

Always cite the exact VSTD version, implementation commit, receipt type, mechanism, and demonstrated test or receipt. Do not turn specification text into an implementation claim.

The bundled checker records a checker verdict. Its historical `independent_audit` field name is not evidence of independence. Claim independent verification only when the receipt’s `independence_basis` demonstrates distinct actors plus the relevant implementation and runtime separation. Matching run results cannot supply that evidence.

6.8 Safe claim template

Using VSTD-Graph-1 at commit `<commit>`, receipt `<receipt-id>` validated the stored provenance graph and replayed its declared graph mechanisms for target `<artifact>`. This establishes receipt integrity and the stated recorded-graph results. It does not establish complete real-world lineage,

legal rights, physical-file identity without a rehashing adapter, or truth outside the declared surface.

VSTD-3 safe template:

Using VSTD-3 at commit `<commit>`, verifier `<mechanism/version>` validated receipt `<receipt-id>` for device/logical boundary `<boundary>` and interval `<interval>`. Claim `<claim>` returned `<PASS|FAIL|UNKNOWN|UNSUPPORTED>` using evidence `<evidence-ids>`. This says `<bounded translation>`. It does not say `<prohibited inference>`.

VSTD-4 safe template:

Using VSTD4-GDC-1 at commit `<commit>`, kernel `<descriptor>` accepted certificate `<digest>` for claim coordinate `<coordinate>` at tier `<tier>` within bounds `<bounds>`. The result was `<PASS|FAIL|UNKNOWN>` and its machine-checkable refutation surface was `<surface>`. This establishes only the grounded decision within the bound roots and exclusions; it does not establish evidence authenticity, complete policy coverage, independent witnessing, or truth outside the coordinate.

VSTD-5 reference-result template:

At receipt `<receipt>`, the VSTD-5 reference mechanism admitted evidence-bound VSTD-4 result `<digest>`, rehashed the embedded witness evidence, and reran the exact seven separation and corroboration mechanisms under `<roots/bounds>`. It returned `<CORROBORATED|REFUTED|UNKNOWN|CONFLICTED>` with conformance `<ESTABLISHED|NOT_ESTABLISHED>`. This does not imply actor trust, an external witness not named by the evidence, a second implementation, or truth outside the checked propositions.

A `CORROBORATED` overall result is valid only with `ESTABLISHED` conformance and `INDEPENDENT` computed separation. If a positive observation survives but any separation seam is unresolved, report overall `UNKNOWN`; preserve `REFUTED` and `CONFLICTED` results rather than softening demonstrated negative evidence.

6.9 Prohibited shortcuts

Do not publish any of these without the missing qualification:

- “VSTD proves provenance.”
- “VSTD proves the dataset is clean.”
- “VSTD proves the license is valid.”
- “VSTD independently verifies itself.”
- “A VSTD-4 result is independently verified” when no separate witnessing seam was demonstrated.
- “UNKNOWN proves no proof exists” or “the next rung is impossible.”
- “VSTD-5 verified” based only on passing the VSTD-5 entry gate.
- “VSTD proves the model is safe.”
- “VSTD verifies the competition result.”
- “Completely verified” when any material residual, valence, mechanism, order, or trust root remains unresolved.
- “Hardware attested” for host inventory, a provider allocation response, or opaque bytes that no implemented verifier authenticated.
- “All compute was accounted for” without complete-mediation evidence for every path in the named governed boundary.
- “No undeclared compute occurred” from a device, host, provider, or fleet receipt.
- “GUILT” from a decorative obligation string, an opaque combined `PASS`, graph placement, actor identity, role, ownership, reputation, or a violation that does not bind the same artifact, scoped obligation, and localized deviation.
- “No GUILT means innocence, exoneration, obligation satisfaction, or no hidden contributor.”
- “Sealed means encrypted, immutable, correct, externally archived, or continuously guarded.”

Chapter 7

Conceptual Foundations, Precedents, and Adjacent Standards

Acronyms: conjunctive normal form (CNF); Certificate Transparency (CT); deletion resolution asymmetric tautology (DRAT); Internet Engineering Task Force (IETF); JavaScript Object Notation (JSON); linear resolution asymmetric tautology (LRAT); National Institute of Standards and Technology (NIST); proof-carrying code (PCC); Principles of Programming Languages (POPL); World Wide Web Consortium provenance vocabulary (PROV); PROV data model (PROV-DM); Protect the Software (PS); Request for Comments (RFC); reverse unit propagation (RUP); Boolean satisfiability problem (SAT); Supply-chain Levels for Software Artifacts (SLSA); satisfiability modulo theories (SMT); SMT library standard (SMT-LIB); The Update Framework (TUF); Verifier Standard (VSTD); World Wide Web Consortium (W3C).

Status: non-normative reader aid

VSTD did not arise in a vacuum, but it also does not inherit another system’s guarantees merely by citing it. This guide separates two jobs:

1. **Orientation links** answer only “what neighboring concept should I recognize?” so an unfamiliar reader can stay in the flow of the guide. On the GitHub Pages site, hovering over or focusing one displays a wiki-style card whose short definition is versioned in this repository. The link itself opens optional Wikipedia background; repository Markdown degrades to that ordinary link and its boundary title. The popup performs no network request, and neither the popup nor the external page is VSTD evidence.
2. **Primary references** support the stated historical or technical precedent by pointing to a standard, specification, or original paper. They do not prove that VSTD is correct, adopted, interoperable, accredited, or conformant to the referenced system.

Implemented commands and supported Python interfaces are documented separately in the generated [command-line interface \(CLI\) and application programming interface \(API\) reference](#). A concept is linked to that reference only when the implementation exposes an exact public coordinate; the guide does not invent an API mapping for a conceptual resemblance.

When a repository orientation definition and a primary source differ, use the primary source for the external concept. When a primary source and a VSTD requirement differ, the VSTD document controls VSTD conformance and the difference must remain explicit.

7.1 Orientation glossary

Concept: Assurance

Optional background: [Information assurance](#)

Repository-owned definition and VSTD boundary: VSTD reports evidence-bounded results, not universal confidence or institutional accreditation.

Concept: TRUST

Optional background: [Proof-carrying code](#)

Repository-owned definition and VSTD boundary: Formal VSTD name for mechanism-earned, artifact-bound support that may move forward only through checked transformations. It is not actor trust, a scalar, or an acronym.

Concept: ROT

Optional background: [Software rot](#)

Repository-owned definition and VSTD boundary: Formal VSTD name for typed, time-indexed degradation of current admissibility. It preserves historical results and is not inferred from age alone. It is not an acronym.

Concept: RUST

Optional background: [Spectrum-based software fault localization](#)

Repository-owned definition and VSTD boundary: Formal VSTD name for inverse-TRUST diagnostic traversal from a descendant deviation toward recorded ancestor candidates. Reachability is not causal localization. It is not an acronym or the Rust programming language.

Concept: Verification complex

Optional background: [Constraint satisfaction problem](#)

Repository-owned definition and VSTD boundary: Named closure coordinates and evidence-bearing relations form the semantic space. Numbered profiles are cumulative requirement formulas over that space, not scalar assurance levels.

Concept: Closure coordinate

Optional background: [Security assurance component](#)

Repository-owned definition and VSTD boundary: One named verification question and failure class. Evidence for one coordinate never supplies another. Closure is always proposition-qualified.

Concept: Numbered profile

Optional background: [Conformance testing](#)

Repository-owned definition and VSTD boundary: VSTD-N and VSTD-Graph-N select cumulative coordinate requirements. Matching object and Graph numbers do not identify the same coordinate.

Concept: Layer and level

Optional background: [Abstraction layer](#)

Repository-owned definition and VSTD boundary: VSTD reserves layer for actual implementation, protocol, or physical stacks and level for named external taxonomies or compatibility identifiers. Neither word is a synonym for a numbered VSTD profile.

Concept: Defense in depth

Optional background: [Defense in depth](#)

Repository-owned definition and VSTD boundary: Multiple independent controls limit the effect of one control failing. VSTD applies the pattern to distinct closure coordinates without claiming that their profile numbers are assurance levels.

Concept: Fail-closed decisions

Optional background: [Fail-safe](#)

Repository-owned definition and VSTD boundary: Missing or exhausted evidence stays UNKNOWN, INDETERMINATE, or UNSUPPORTED; it does not become a pass.

Concept: Trusted computing base

Optional background: [Trusted computing base](#)

Repository-owned definition and VSTD boundary: Every result must expose the mechanism and trust roots on which it depends.

Concept: Zero trust

Optional background: [Zero trust architecture](#)

Repository-owned definition and VSTD boundary: VSTD borrows no product architecture wholesale; it uses explicit verification rather than identity or location as an automatic correctness signal.

Concept: Canonicalization

Optional background: [Canonicalization](#)

Repository-owned definition and VSTD boundary: Stable fields need one declared byte representation before hashing. VSTD's formats are not thereby RFC 8785 implementations. Public API: `compute_canonical_digest` (reference).

Concept: Content addressing

Optional background: [Content-addressable storage](#)

Repository-owned definition and VSTD boundary: Artifact and receipt coordinates bind declared bytes through digests; a digest alone does not establish origin or truth.

Concept: Cryptographic digest

Optional background: [Cryptographic hash function](#)

Repository-owned definition and VSTD boundary: Hash observations can establish byte identity within an algorithm and observation boundary, not semantic correctness.

Concept: Provenance

Optional background: [Data provenance](#)

Repository-owned definition and VSTD boundary: VSTD-Graph records declared entities, transformations, and ancestry while preserving incomplete or unauthenticated history as such.

Concept: Hypergraph

Optional background: [Hypergraph](#)

Repository-owned definition and VSTD boundary: N-ary transformation edges preserve many-input and many-output structure without flattening it into ambiguous binary links.

Concept: Attestation

Optional background: [Attestation](#)

Repository-owned definition and VSTD boundary: VSTD-3 records who or what supplied evidence, the mechanism used, and the resulting evidence ceiling.

Concept: Trust root

Optional background: [Trust anchor](#)

Repository-owned definition and VSTD boundary: A declared root is an explicit dependency and stopping boundary, not evidence that the root is honest.

Concept: Reproducibility

Optional background: [Reproducibility](#)

Repository-owned definition and VSTD boundary: VSTD binds the exact mechanism, inputs, environment, and equivalence relation required by the claim rather than treating the word as self-defining. Public API: `ReproducibilityLevel` (reference).

Concept: Reproducible build

Optional background: [Reproducible builds](#)

Repository-owned definition and VSTD boundary: Recreating identical artifacts is an important special case of portable checking, not a proof of every property of the artifact or of distinct actors.

Concept: Falsifiability

Optional background: [Falsifiability](#)

Repository-owned definition and VSTD boundary: VSTD-4 requires an explicit, bounded way for an outside checker to refute the exact claim. It does not turn Popper’s philosophy into a software theorem.

Concept: Proof-carrying artifact

Optional background: [Proof-carrying code](#)

Repository-owned definition and VSTD boundary: The engineering precedent is that an untrusted producer can ship a result with a smaller consumer-checkable certificate under a declared policy. Public API: `DecisionCertificate` (reference).

Concept: SAT

Optional background: [Boolean satisfiability problem](#)

Repository-owned definition and VSTD boundary: The reference subset encodes finite admission questions; SAT success establishes only the encoded formula.

Concept: CNF

Optional background: [Conjunctive normal form](#)

Repository-owned definition and VSTD boundary: VSTD’s bounded policy encodings use finite CNF and do not equate arbitrary CNF with 3-SAT.

Concept: Resolution

Optional background: [Resolution](#)

Repository-owned definition and VSTD boundary: Clausal refutations provide checkable evidence for an unsatisfiable result within the implemented proof format.

Concept: Unit propagation

Optional background: [Unit propagation](#)

Repository-owned definition and VSTD boundary: The minimal trusted checker validates the supported reverse-unit-propagation certificate path rather than trusting the producer’s solver.

Concept: Three-valued result

Optional background: [Three-valued logic](#)

Repository-owned definition and VSTD boundary: UNKNOWN is a first-class refusal to overstate, not a Boolean false and never a pass. VSTD’s statuses are not claimed to implement one historical three-valued logic.

Concept: Append-only transparency

Optional background: [Certificate Transparency](#)

Repository-owned definition and VSTD boundary: Immutable receipts and additive corrections share an auditability goal with append-only logs; VSTD is not a Certificate Transparency implementation.

Concept: Update freshness

Optional background: [The Update Framework](#)

Repository-owned definition and VSTD boundary: Staleness, rollback, revocation, and key compromise are separate from content integrity and require explicit current-state evidence.

Concept: Semantic versioning

Optional background: [Semantic Versioning](#)

Repository-owned definition and VSTD boundary: Repository releases use semantic versions independently of the VSTD object and Graph profile numbers.

Concept: Object language and metalanguage

Optional background: [Metalogic](#)

Repository-owned definition and VSTD boundary: VSTD uses this only as a design analogy for examining a verification surface; it does not claim that every adjacent profile is a formal metalanguage.

Concept: Undefinability of truth

Optional background: [Tarski's undefinability theorem](#)

Repository-owned definition and VSTD boundary: The verification complex expressly does not derive its architecture or observational limits from Tarski's theorem.

7.2 Primary reference map

VSTD design seam: Separate failure controls and fail-safe defaults

Primary or official reference: Saltzer and Schroeder, *The Protection of Information in Computer Systems* (1975)

Relevant precedent and explicit limit: Classic security-design principles include fail-safe defaults, complete mediation, separation of privilege, least privilege, and least common mechanism. They motivate separating failure surfaces; they do not derive VSTD's five coordinates on either axis.

VSTD design seam: Security-assurance components and packages

Primary or official reference: Common Criteria, [Part 3: Security assurance components](#) (CC:2022 Revision 1)

Relevant precedent and explicit limit: Established precedent for decomposing assurance into named components and packages. VSTD is not Common Criteria, accredited evaluation, or an Evaluation Assurance Level.

VSTD design seam: Canonical JSON as cryptographic input

Primary or official reference: IETF Independent Stream, [RFC 8785: JSON Canonicalization Scheme](#)

Relevant precedent and explicit limit: Shows why cryptographic operations over JSON require invariant representation. VSTD uses its own declared canonicalization rules and must not claim RFC 8785 conformance unless a format actually implements it.

VSTD design seam: Provenance entities, activities, and agents

Primary or official reference: W3C, [PROV-DM: The PROV Data Model](#)

Relevant precedent and explicit limit: Standardized vocabulary and constraints for interoperable provenance. VSTD-Graph’s artifact and transformation model is adjacent, not a PROV implementation or complete history claim.

VSTD design seam: Supply-chain step and artifact attestations

Primary or official reference: [in-toto specification v1.0](#) and [SLSA v1.2](#)

Relevant precedent and explicit limit: Established formats and levels for materials, products, builders, steps, and provenance. VSTD may bind their outputs as evidence but does not manufacture their authorization or assurance level.

VSTD design seam: Release preservation and provenance integrity

Primary or official reference: NIST, [Special Publication \(SP\) 800-218: Secure Software Development Framework 1.1](#)

Relevant precedent and explicit limit: Protect the Software practices PS.3.1 and PS.3.2 cover archiving releases, maintaining provenance, protecting its integrity, and enabling recipient verification. This is operational precedent, not VSTD certification.

VSTD design seam: Independent recreation of artifacts

Primary or official reference: Reproducible Builds, [formal definition](#)

Relevant precedent and explicit limit: Defines the source, environment, instruction, and artifact relationship needed for bit-for-bit recreation. VSTD permits other explicitly declared equivalence relations and does not infer truth from reproducibility alone.

VSTD design seam: Producer-supplied, consumer-checked certificates

Primary or official reference: Necula, [Proof-Carrying Code](#) (POPL 1997)

Relevant precedent and explicit limit: Primary precedent for an untrusted producer supplying a proof checked under a defined policy by the consumer. VSTD generalizes the receipt pattern but does not inherit PCC’s safety theorem.

VSTD design seam: Checkable SAT refutations

Primary or official reference: Wetzler, Heule, and Hunt, [DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs](#) (2014)

Relevant precedent and explicit limit: Demonstrates checking unsatisfiability proofs outside the solver rather than trusting its answer. VSTD’s implemented certificate is a narrower declared RUP path, not arbitrary DRAT.

VSTD design seam: Explicit indeterminate solver results

Primary or official reference: [SMT-LIB Standard 2.7](#)

Relevant precedent and explicit limit: The standard response grammar includes `sat`, `unsat`, and `unknown`. VSTD’s richer status vocabulary is independently defined, but the refusal to fabricate a Boolean answer has established solver precedent.

VSTD design seam: Append-only evidence and independently detectable equivocation

Primary or official reference: IETF, [RFC 9162: Certificate Transparency Version 2.0](#)

Relevant precedent and explicit limit: Merkle inclusion and consistency proofs support auditing an append-only log, while the RFC also names split-view limitations. VSTD’s additive history is analogous but not a CT log. ## LRAT and DRAT

Competition clausal proof formats—such as deletion resolution asymmetric tautology (DRAT) and linear resolution asymmetric tautology (LRAT)—answer whether a propositional formula is unsatisfiable. In the [primary reference map](#), VSTD cites Wetzler, Heule, and Hunt (2014) for checkable clausal refutations. VSTD’s grounded decision certificates build on this precedent by coupling clausal refutations directly to semantic claim groundings, ensuring a valid proof cannot certify an ungrounded formula.

7.3 How to cite these precedents

Use language such as:

- “VSTD’s portable-certificate design is adjacent to proof-carrying code.”
- “VSTD-Graph overlaps W3C PROV, in-toto, and SLSA at the provenance boundary.”
- “The refusal to convert resource exhaustion into a false result has precedent in the `unknown` response of SMT-LIB.”

Do not write:

- “Saltzer and Schroeder prove the VSTD verification complex.”
- “VSTD implements PROV, SLSA, in-toto, TUF, Common Criteria, or Certificate Transparency,” unless separately demonstrated by a named conformance mechanism.
- “These citations establish VSTD’s security, completeness, adoption, or novelty.”

The point of the map is traceable intellectual context: which established problem a VSTD rule resembles, where the design deliberately differs, and what remains original project architecture rather than inherited authority.

Chapter 8

Platform Interoperability and Hardware Envelopes

Acronyms: 64-bit Arm instruction-set architecture (ARM64); application programming interface (API); Concise Binary Object Representation (CBOR); command-line interface (CLI); CBOR Object Signing and Encryption (COSE); continuous integration (CI); Java unit test report format (JUnit); JavaScript Object Notation (JSON); operating system (OS); reduced instruction set computer (RISC); Supply Chain Integrity, Transparency, and Trust (SCITT); Verifier Standard (VSTD); Windows Subsystem for Linux 2 (WSL2); ZIP archive format (ZIP).

This document records where the Verifier Standard (VSTD) reference repository has platform evidence and where it does not. Its machine-checked reference-component table is separate from the broader evidence-surface summary. Neither is a declaration that every verifier, prover, solver, adapter, or dependency works on every operating system (OS).

8.1 Evidence coordinates and freshness

Three surfaces must stay separate: source-controlled **test intent**, an exact hosted **observation**, and the **candidate being evaluated now**. The generated component table below records intent. Its `CONFIGURED_UNRUN` labels do not assert that a component has never run; they cannot be promoted to support claims inside a source manifest.

For the current candidate, select its successful **repository-checks** run from the [workflow history](#). This is a discovery link, not immutable evidence. Require the exact head, base, executed checkout, run identifier and attempt in the retained reports. Retargeting a request or retaining an earlier green check does not establish fresh integration. Publication additionally requires a successful push run on the exact final **main** commit, as specified in **the release procedure**.

A changed implementation, catalog, test, workflow or dependency requires reassessment; the prior run remains an observation of its own coordinate. Native results do not inherit another component’s tests, and recorded environment metadata is not hardware or runtime attestation.

8.2 Recorded candidate observation

[Run 34077770237](#), attempt 1, completed successfully for head `b6b1a82b862219ffc3c44bb2f4b544444d921715`, base `499c0613c93232628abfdeb497d907549fd593fc`, and executed pull-request merge `884b3cfc9dd36ac0914a5482bece0c13abf33f64`. The executed and head trees both equal `5135df58006e602606e9a8f3`

04ac43d4e83d186e. This is a pre-retarget observation, not evidence for a later candidate or the final release commit.

All four component reports were reconstructed byte-for-byte from their retained raw environment and JUnit documents with the repository builder. That is reproducibility of report construction, not a second native execution or an independently implemented checker. Each environment records CPython 3.12.10; all four record zero failures/errors.

Recorded coordinate: Linux x86-64 / Ubuntu 24.04

Passed test instances: 1,114

Skipped test instances: 1

Skipped collection placeholders: 1

Recorded coordinate: Windows x86-64 / Windows Server 2025

Passed test instances: 1,109

Skipped test instances: 6

Skipped collection placeholders: 1

Recorded coordinate: Intel macOS / macOS 15

Passed test instances: 1,114

Skipped test instances: 1

Skipped collection placeholders: 1

Recorded coordinate: Apple ARM64 macOS / macOS 15

Passed test instances: 1,114

Skipped test instances: 1

Skipped collection placeholders: 1

Collection placeholders are not executed tests. Parameterized instances are not counts of independent mechanisms. The reports map 18 catalog entries through 13 test modules and 413 mapped instances; shared tests and wrappers do not become independent verifiers. The raw reports retain the exact skip reasons. Unavailable optional mechanisms and platform-specific filesystem cases do not earn passing coverage. See `TEST_SKIP_RUBRIC.md` for the normative skip classification rubric.

8.3 Evidence classes

- **Hosted execution:** a CI job executes the named component on the named OS. A successful exact run is still bounded by its runner image and inputs.
- **Packaging only:** the distribution is built and checked on that OS; this does not exercise every installed mechanism.
- **Static or artifact-only:** tests inspect source, configuration, or recorded bytes without executing the native mechanism.
- **Documented local execution:** the repository records a local run and reproduction procedure, but CI does not currently repeat it.
- **Unsupported by this repository path:** the checked-in procedure does not offer a native path for that OS. This says nothing about what an upstream project might support.

- **Unknown:** no current repository evidence establishes support or incompatibility.

8.4 Evidence-surface summary

The observations in this table refer only to the recorded candidate above. For another head, apply the freshness rules and inspect its own run. A complete-suite pass covers collected, non-skipped cases, not every possible behavior of every exported function.

Surface: Python reference contracts

Recorded execution or packaging scope: Python 3.10–3.13 on Linux; complete Python 3.12 suite on Linux, Windows, Intel macOS and Apple ARM64 macOS

Boundary: Exact tests and optional profiles, with skips retained; no universal native-adapter qualification.

Surface: Generic-run capture and platform comparison

Recorded execution or packaging scope: Linux, Windows and Intel macOS run the same portable specimen; an aggregate compares their declared receipt results

Boundary: Isolates the operating-system dimension only when non-platform bindings match; Apple ARM64 contract tests do not establish cross-architecture equivalence.

Surface: Release artifact byte reproducibility

Recorded execution or packaging scope: Builds on Linux, Windows and macOS; aggregate compares the artifact sets

Boundary: Packaging reproducibility, not runtime equivalence.

Surface: Installed-wheel smoke

Recorded execution or packaging scope: Selected public commands exercised on Linux outside the source checkout

Boundary: Not an installed-wheel claim for every platform or command.

Surface: Core receipts, geometry, Graph, evidence replay, catalog, packages, planning and workflow

Recorded execution or packaging scope: Their collected Python cases are included in the four-coordinate suite

Boundary: Each native result retains its scope; planning and workflow metadata do not become execution or conformance.

Surface: SCITT cryptographic example

Recorded execution or packaging scope: Optional profile exercised on all four coordinates using local test keys and a local test log

Boundary: Signature and inclusion checking do not establish public registration, payload truth, independent implementations or issuer authority.

Surface: Artifact freeze, finite seal and thaw

Recorded execution or packaging scope: Optional cryptographic profile exercised on all four coordinates

Boundary: Filesystem-specific skips remain excluded; no privileged-write prevention, trusted time or general network-filesystem guarantee.

Surface: RISC Zero native prover/verifier

Recorded execution or packaging scope: Hosted checks inspect source and recorded artifacts, not a fresh native proof execution; the documented local path is Linux x86-64 under WSL2

Boundary: No native Windows path or established macOS support from this repository evidence; no external witness-truth or VSTD receipt-mapping claim.

Surface: Other native prover, verifier, solver, hardware or vendor adapters

Recorded execution or packaging scope: UNKNOWN without their own exact mechanism and execution evidence

Boundary: No support is inherited from a descriptor, wrapper, package or unrelated Python test.

8.5 Machine-checked reference-component contract intent

The new **stored component format** is also exercised by the complete-suite jobs, including a fixed canonical-digest specimen, exact binary-byte round trips and package-selected planning. The recorded observation establishes only those executed cases at its named coordinate; changed candidates require their own reports. Portable storage does not qualify a retained native implementation. Storage is supporting infrastructure, not an extra independent verifier counted in the 43-component inventory below.

The six newly represented experimental families contribute 12 role-specific entries: source grounding, relation-boundary assessment, actual deriver self-status, global-cycle assessment, source-grounded bounded completeness, and runtime-authority correspondence. Assessment or bounded recording and receipt rechecking remain separate roles, not independent implementations. Their native declaration and receipt formats remain direct-submodule experimental contracts; this source-intent table does not add stable wire identifiers. Stored-package detection and planning match exact identifiers only. Readiness does not invoke an endpoint, grant authorization, or close ordinary or self-closure geometry, and missing native inputs or result mappings remain NOT_ESTABLISHED.

The source-controlled **platform-component-contracts.json** manifest enumerates exactly the current reference catalog and four separate execution coordinates. CONFIGURED_UNRUN records configured intent only; it is not hosted execution, compatibility, or support evidence. BEHAVIOR means the mapped tests call the component on bounded positive or adversarial cases. It never means the tests cover every native input, dependency, filesystem, trust root, or external implementation.

Intel macOS cryptographic coverage is an experimental source-build target. The pinned `cryptography==50.0.0` dependency follows the upstream [removal of Intel macOS support](#). The Intel job explicitly builds `cryptography` and `cbor2` from source with the hosted runner's Rust and C compilers and Homebrew OpenSSL 3, following the upstream [source-build prerequisites](#). It must pass installation and the complete suite before emitting a report. A successful run supplies only evidence for that exact build and runner; it does not restore upstream support or qualify other Intel macOS environments. Apple ARM64 is a separate coordinate.

Raw environment and JUnit files are boundary-checked before artifact upload, including after a failing test command. Missing or prohibited evidence fails the job and is not uploaded. The gate does not redact or rewrite the evidence bytes. The explicit `scripts.pytest_public_evidence` plugin records skip source

locations relative to the declared test root before serialization. It preserves outcomes, reasons, and line numbers; outside-root locations remain unchanged and subject to the upload gate.

After the complete test command succeeds, CI emits a separate, platform-neutral **platform-component-contract.json** artifact. Its artifact name includes the workflow run identifier, run attempt, and platform coordinate. That runtime report binds the exact Git and workflow coordinate, workflow-file digest, test command, runner image, interpreter, catalog version and digest, source manifest digest, the exact raw environment-document digest, JUnit result, component identifiers, test modules, dependency profiles, and bounded test scopes. The builder also requires at least one successful, non-skipped test case from every mapped module and rejects malformed or contradictory JUnit suite counts; a green unrelated subset or summary-only failure cannot populate all component rows. The environment loader rejects unknown or duplicate fields before binding the raw document bytes. The raw environment record is uploaded after later-step failures, and the JUnit report is included whenever the test command produced it. The component report is not created or uploaded after failure. These hosted artifacts are retained for 30 days; their presence during that window is not the long-term release record.

For a release, the owner supplies the exact successful **repository-checks** push-run identifier. The release workflow requires that run to target the tagged commit on the protected default branch and contain one successful **conformance-gate**. It downloads all four component reports plus their raw environment and JUnit evidence, independently reconstructs every report from those raw inputs, and rejects a missing coordinate, mixed run or attempt, pull-request synthetic merge, changed workflow/catalog/manifest, failed or contradictory test evidence, altered environment bytes, or byte-altered report. The resulting deterministic platform-evidence ZIP contains an internal manifest binding every member digest and byte length to the release tag, repository, commit, run, attempt, and bounded claim. The ZIP is boundary-scanned, attested, and attached to the immutable release. It retains **verification_effect = NONE**: durable evidence does not strengthen what the mapped tests establish.

Reference component: Artifact freeze and seal verifier; **component:artifact-bundle-verifier**

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_artifact_control.py

Dependency profile: seal, test

Bounded test scope: Exercise exact-byte freeze, finite cryptographic seal, external-anchor, verification, and thaw behavior. Passing does not establish privileged-write prevention, trusted time, or untested filesystem semantics.

Reference component: Experimental finite authority-composition assessor; **component:artifact-network-authority-composition-assessor**

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_authority_composition.py; tests/test_authority_composition_catalog.py

Dependency profile: test

Bounded test scope: Exercise exact member/composite commit and authority-model binding, bounded reachable asynchronous-interleaving correspondence, distinct negative and unknown preservation, and inert package-to-index-to-planning/readiness integration. Passing does not establish source grounding, self-derivation, stronger completeness, runtime correspondence, general agency, TypeScript support, or a six-axis status upgrade.

Reference component: Experimental source-grounded bounded-completeness assessor; **component:** artifact-network-bounded-completeness-assessor

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_bounded_completeness.py; tests/test_mechanism_catalog.py

Dependency profile: test

Bounded test scope: Exercise exact finite denominator, observation, source-grounding, member-receipt and evidence-budget binding plus inert package-to-index-to-planning/readiness integration. Passing does not establish an exhaustive real-world denominator, source truth, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental bounded-completeness receipt rechecker; **component:** artifact-network-bounded-completeness-receipt-rechecker

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_bounded_completeness.py; tests/test_mechanism_catalog.py

Dependency profile: test

Bounded test scope: Exercise exact canonical bounded-completeness receipt replay with negative, incomplete, invalid and unknown states preserved, plus inert package-bound detection, planning and readiness. Passing establishes neither an independently implemented checker nor source truth, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental strict silo-composition declaration decoder; **component:** artifact-network-composition-declaration-decoder

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_silo_composition_fixture.py; tests/test_silo_composition_receipt.py

Dependency profile: test

Bounded test scope: Exercise exact verifier-silo-composition-1 declaration decoding, including canonical member ordering, unique bounded commit digests, strict fields, and optional composite binding. Passing does not resolve commit bytes, check general graph edges, establish TypeScript support, or grant composition admissibility.

Reference component: Experimental silo-composition assessment receipt builder; **component:** artifact-network-composition-receipt-builder

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_silo_composition_fixture.py; tests/test_silo_composition_receipt.py

Dependency profile: test

Bounded test scope: Exercise construction of a declaration-, mechanism-, member-receipt-, optional-composite-, and assessment-bound verifier-silo-composition-assessment-receipt-1 record from current local stores. Passing does not provide an independently implemented checker, check general graph edges, establish TypeScript support, or prove artifact truth.

Reference component: Experimental silo-composition assessment receipt rechecker; **component:** artifact-network-composition-receipt-rechecker

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_silo_composition_fixture.py; tests/test_silo_composition_receipt.py

Dependency profile: test

Bounded test scope: Exercise strict receipt decoding and independent full recomputation against the exact declaration, mechanism bytes, resolved member silos, optional composite silo, nested receipts, typed assessment, and canonical bytes. Passing establishes agreement with the same Python reference mechanism only; it does not establish implementation independence, a general graph-edge checker, TypeScript support, or artifact truth.

Reference component: Experimental actual deriver self-status rechecker; `component:artifact-netwo`
`rk-deriver-self-status-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_deriver_self_status.py`; `tests/test_mechanism_catalog.py`

Dependency profile: test

Bounded test scope: Exercise exact executable, runtime coordinate, invocation, input and transcript re-execution binding plus inert package-to-index-to-planning/readiness integration. Passing does not establish platform sandboxing, independent implementation, actor independence, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental bounded deriver session recorder; `component:artifact-netwo`
`rk-deriver-session-recorder`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_deriver_self_status.py`; `tests/test_mechanism_catalog.py`

Dependency profile: test

Bounded test scope: Exercise bounded subprocess invocation and exact executable, runtime coordinate, input and transcript capture plus inert package-to-index-to-planning/readiness integration. Recording is live execution only when explicitly invoked; discovery and readiness do not invoke it. Passing does not establish sandboxing, independent verification, authorization, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental selected finite silo-composition qualifier; `component:artifact-netwo`
`rk-finite-composition-qualifier`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_composition_qualification.py`; `tests/test_composition_qualification`
`_catalog.py`

Dependency profile: test

Bounded test scope: Exercise exact selected local declaration, member/composite commit and store bindings, separate legacy and finite recomputation, qualification intersection, negative and UNKNOWN preservation, and inert package-to-index-to-planning/readiness integration. Passing does not establish an atomic snapshot, portable replay receipt, source proof, runtime/model correspondence, general composed agency, automatic execution, TypeScript support, universal platform support, or a six-axis status upgrade.

Reference component: Experimental grounded global-cycle assessor; `component:artifact-netwo`
`rk-global-cycle-assessor`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_global_cycle_assessment.py`; `tests/test_mechanism_catalog.py`

Dependency profile: test

Bounded test scope: Exercise relation-receipt and semantic-source grounding, ordinary support-cycle, directionless simultaneous-constraint, and explicit backward-time facets under finite work bounds, plus inert package-bound integration. Passing does not infer physical causation or correctness, grant authorization, provide automatic execution, close verification geometry, establish TypeScript support, or establish universal platform support.

Reference component: Experimental global-cycle receipt rechecker; `component:artifact-netwo`
`rk-global-cycle-receipt-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_global_cycle_assessment.py`; `tests/test_mechanism_catalog.py`

Dependency profile: test

Bounded test scope: Exercise exact global-cycle receipt reproduction against retained relation and grounded semantic evidence, including substitution and finite-work refusal, plus inert package-bound integration. Passing establishes neither an independent checker nor physical causation, correctness, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental finite-set proposition-transfer assessor; `component:artifact-netwo`
`rk-proposition-transfer-assessor`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_proposition_transfer.py; tests/test_proposition_transfer_catalog.py; tests/test_proposition_transfer_fixture.py

Dependency profile: test

Bounded test scope: Exercise exact declaration, commit, census, artifact and context binding, bounded finite-set predicates and transfer, profile mismatch, negative and unknown preservation, and package-to-index-to-planning/readiness integration. Passing does not establish runtime availability from a catalog match, preserved agency, general deduction, six-axis silo completeness, TypeScript support, or universal platform support.

Reference component: Experimental finite-set proposition-transfer receipt rechecker; `component:artifact-network-proposition-transfer-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_proposition_transfer.py; tests/test_proposition_transfer_catalog.py; tests/test_proposition_transfer_fixture.py

Dependency profile: test

Bounded test scope: Exercise exact complete semantic receipt recomputation using the same Python reference mechanism, including reproduced negative and unknown fields, declaration/profile binding refusal, replay mismatch distinct from negation, and nonexecuting package-bound planning. Passing does not establish independent implementation, execution identity, preserved agency, general deduction, six-axis silo completeness, TypeScript support, or universal platform support.

Reference component: Experimental relation-boundary assessor; `component:artifact-network-relation-boundary-assessor`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_mechanism_catalog.py; tests/test_relation_boundary.py

Dependency profile: test

Bounded test scope: Exercise exact source, target, translation mechanism, topology, facet preservation and loss-map binding with source-grounded evidence, plus inert package-to-index-to-planning/readiness integration. Passing does not establish source truth, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental relation-boundary receipt rechecker; `component:artifact-network-relation-boundary-receipt-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_mechanism_catalog.py`; `tests/test_relation_boundary.py`

Dependency profile: test

Bounded test scope: Exercise exact relation-boundary receipt reproduction against retained source, target, relation and grounding evidence while preserving information loss and unresolved states, plus inert package-bound integration. Passing establishes neither an independent checker nor source truth, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental runtime-authority correspondence assessor; `component:artifact-network-runtime-authority-correspondence-assessor`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_mechanism_catalog.py`; `tests/test_runtime_authority_correspondence.py`

Dependency profile: test

Bounded test scope: Exercise exact authority model, runtime coordinate, retained trace, transition denominator, actual deriver-session output and finite reconstruction binding, plus inert package-bound integration. Passing does not establish general agency, complete transition coverage beyond the declared denominator, runtime mediation, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental runtime-authority correspondence receipt rechecker; `component:artifact-network-runtime-authority-correspondence-receipt-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_mechanism_catalog.py`; `tests/test_runtime_authority_correspondence.py`

Dependency profile: test

Bounded test scope: Exercise exact runtime-authority receipt reconstruction from retained declaration, model, trace and deriver-session evidence while rejecting forged result fields, plus inert package-bound integration. Passing establishes neither an independent implementation nor general agency, runtime mediation, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental source-grounding assessor; `component:artifact-network-source-grounding-assessor`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_mechanism_catalog.py`; `tests/test_source_grounding.py`

Dependency profile: test

Bounded test scope: Exercise exact source bytes, semantic frame, interpretation, proposition, certificate and checker-coordinate binding plus inert package-to-index-to-planning/readiness integration. Passing establishes only the supported finite predicate over retained bytes; it does not establish source truth, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental source-grounding receipt rechecker; `component:artifact-network-source-grounding-receipt-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_mechanism_catalog.py`; `tests/test_source_grounding.py`

Dependency profile: test

Bounded test scope: Exercise exact source-grounding receipt reproduction from the declaration and retained source and certificate evidence while preserving unsupported and failed predicates, plus inert package-bound integration. Passing establishes neither an independent checker nor source truth, authorization, automatic execution, closed verification geometry, TypeScript support, or universal platform support.

Reference component: Experimental artifact-network transfer materializer; `component:artifact-network-transfer-materializer`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_artifact_network.py

Dependency profile: seal, test

Bounded test scope: Exercise canonical signed-transfer parsing, exact object closure, publisher and key continuity, signature validation, assessment recomputation, native export materialization, cloning, rebuilding, and adversarial substitution and bound rejection. A successful hosted run establishes only those mechanisms at its exact recorded runner, interpreter, source, catalog, and manifest coordinates; it does not establish a deployed endpoint, publisher identity, artifact truth, external interoperability, or untested platform behavior.

Reference component: Experimental finite typed-formation certificate checker; `component:artifact-network-typed-formation-checker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_formation_catalog.py; tests/test_formation_resources.py; tests/test_formation_storage.py; tests/test_typed_formation.py

Dependency profile: test

Bounded test scope: Exercise independent finite constructor checking, exact subject and certificate binding, ordered path endpoints, retained quotation history, malformed records, unsupported profiles, bounds, inert package-to-planning integration, stored-silo coordinate binding and separate census-byte retention. Passing does not establish source self-derivation, completeness, agency preservation, TypeScript support or universal platform support.

Reference component: Experimental finite typed-formation certificate producer; `component:artifact-network-typed-formation-producer`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_formation_catalog.py; tests/test_formation_resources.py; tests/test_typed_formation.py

Dependency profile: test

Bounded test scope: Exercise bounded finite proof-step production and independent rechecking, exact compiled profile and packaged resources, failure and unsupported interpretation, and inert package-to-planning integration. Producing a certificate does not establish source self-derivation, completeness, agency preservation, TypeScript support or universal platform support.

Reference component: Experimental finite-formation receipt rechecker; `component:artifact-network-typed-formation-receipt-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_formation_catalog.py`; `tests/test_formation_receipt.py`; `tests/test_formation_receipt_resources.py`

Dependency profile: test

Bounded test scope: Exercise exact selection, actual observed census bytes including substitutions, complete canonical replay, preserved negative and unknown reports, hostile input bounds, packaged resources and nonexecuting package-bound planning/readiness. Passing does not establish source self-derivation, stronger completeness, grounding, agency preservation, independent execution or universal platform support.

Reference component: Experimental typed-formation native session adapter; `component:artifact-network-typed-formation-session-mechanism`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_formation_catalog.py`; `tests/test_formation_mechanism.py`; `tests/test_session_binding_snapshots.py`; `tests/test_typed_formation.py`

Dependency profile: test

Bounded test scope: Exercise exact native proposition, implementation, predicate, context, assumption roots, ordered byte references and execution bounds, caller and invocation mutation refusal, and nonexecuting package-bound readiness. Passing does not establish source self-derivation, completeness, agency preservation, TypeScript support or universal platform support.

Reference component: Experimental composed-graph untraversability analyzer; `component:composed-untraversability-analyzer`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_untraversable.py`; `tests/test_untraversable_graph_binding.py`

Dependency profile: test

Bounded test scope: Exercise observer-relative composed closure, conjunctive inference, exact witnesses, evidence and resource uncertainty, conflict preservation, separate awareness modes, and graph substitution refusal. Passing establishes neither universal non-inferability, runtime mediation, authorization, identity, nor intent.

Reference component: VSTD-1 generic-run receipt validator; `component:generic-run-validator`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_public_cli.py`

Dependency profile: test

Bounded test scope: Execute the bounded command fixture, validate and inspect its receipt, and rerun it through the public command-line interface.

Reference component: Experimental exact-graph-bound topology analyzer; `component:graph-topology-analyzer`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_graph_topology.py`; `tests/test_graph_topology_integration.py`

Dependency profile: test

Bounded test scope: Exercise exact graph and port binding, structural cycles without inferred contradiction, bounded simultaneous classical Boolean equations, temporal-offset consistency per clock and unit, and public planning integration. Passing does not establish physical causality, runtime control, assurance, or external verifier qualification.

Reference component: Bounded operating-system result comparator; `component:platform-run-comparator`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_platform_comparison.py`

Dependency profile: test

Bounded test scope: Exercise positive, conflicted, incomplete, malformed, and binding-drift platform-comparison cases over recorded receipts.

Reference component: Recorded provenance policy verifier; `component:provenance-policy-verifier`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_public_data.py

Dependency profile: test

Bounded test scope: Exercise provenance-policy acceptance and rejection over bounded data-receipt fixtures.

Reference component: SCITT native-evidence consumer; `component:scitt-evidence-consumer`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_scitt_crypto_example.py; tests/test_scitt_interop.py

Dependency profile: scitt, test

Bounded test scope: Exercise semantic SCITT evidence mapping plus the local-key cryptographic example. Passing does not establish public transparency infrastructure or production trust roots.

Reference component: VSTD and SCITT adjacent-result composer; `component:scitt-result-composer`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_scitt_crypto_example.py; tests/test_scitt_interop.py

Dependency profile: scitt, test

Bounded test scope: Exercise bounded adjacent-result composition, mismatch preservation, and the local-key cryptographic example without strengthening either native result.

Reference component: VSTD-Graph assurance-log rechecker; `component:vstd-graph-assurance-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_evidence_bound_assurance.py`

Dependency profile: test

Bounded test scope: Exercise bounded assurance-log replay, dependency handling, and failure preservation.

Reference component: Evidence-bound VSTD-Graph profile rechecker; `component:vstd-graph-level-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_evidence_bound_assurance.py`

Dependency profile: test

Bounded test scope: Exercise evidence-bound Graph-level record rechecking and unresolved-evidence behavior.

Reference component: VSTD-Graph receipt validator; `component:vstd-graph-receipt-validator`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_public_data.py`

Dependency profile: test

Bounded test scope: Exercise bounded Graph receipt validation, malformed input rejection, and policy interactions.

Reference component: VSTD-1 satisfiability and grounding auditor; `component:vstd1-independent-auditor`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_core_receipt_integrity.py

Dependency profile: test

Bounded test scope: Exercise the bundled satisfiability and grounding audit while constructing adversarial receipt-integrity fixtures.

Reference component: Strict VSTD-2 geometry loader and semantic validator; **component:**vs
td2-geometry-loader

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_geometry_io.py

Dependency profile: test

Bounded test scope: Exercise valid geometry loading, canonical normalization, and adversarial structural and numeric rejection.

Reference component: VSTD-3 hardware receipt validator; **component:**vstd3-receipt-validator

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_vstd3_emulator.py

Dependency profile: test

Bounded test scope: Exercise hardware-receipt validation using deterministic emulated evidence; no physical-device or vendor attestation is established.

Reference component: VSTD-4 evidence-bound receipt rechecker; **component:**vstd4-evidence
-rechecker

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_evidence_bound_assurance.py

Dependency profile: test

Bounded test scope: Exercise evidence-bound receipt replay, mechanism binding, and non-establishment when evidence or mechanisms drift.

Reference component: VSTD-4 grounded decision certificate kernel; `component:vstd4-grounded-certificate-kernel`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_public_cli.py

Dependency profile: test

Bounded test scope: Exercise grounded certificate acceptance and refutation through the deterministic public demonstration scenarios.

Reference component: VSTD-4 reverse-unit-propagation proof checker; `component:vstd4-refutation-checker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_refutation_certificate.py

Dependency profile: test

Bounded test scope: Exercise accepted and rejected reverse-unit-propagation certificates without inferring formula grounding or external truth.

Reference component: Bounded VSTD-4 refutation proof producer; `component:vstd4-refutation-producer`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: tests/test_refutation_certificate.py

Dependency profile: test

Bounded test scope: Exercise satisfiable, unsatisfiable, and resource-bounded proof search with independently checked emitted refutations.

Reference component: VSTD-5 witness corroboration rechecker; `component:vstd5-witness-rechecker`

Linux x86-64: CONFIGURED_UNRUN

Windows x86-64: CONFIGURED_UNRUN

macOS Intel x86-64: CONFIGURED_UNRUN

macOS Apple ARM64: CONFIGURED_UNRUN

Coverage kind: BEHAVIOR

Test modules: `tests/test_evidence_bound_assurance.py`

Dependency profile: `test`

Bounded test scope: Exercise witness rechecking while preserving refuted, unknown, conflicted, and independence-limited outcomes.

8.6 Reading a platform-comparison result

The comparator classifies supplied recorded results; it does not certify an OS or a component in the abstract:

- **PASS** means the complete declared receipt set had matching bound non-platform coordinates and matching declared result projections.
- **CONFLICTED** means comparable receipts disagreed on a declared result surface.
- **NOT_ESTABLISHED** means required coverage or comparability evidence was absent or did not isolate the OS dimension.
- **INVALID** means an input was malformed, canonically inconsistent, or internally contradictory.

A mechanism may be cross-platform compatible even when this repository reports **UNKNOWN**; the missing fact is evidence, not necessarily capability. Conversely, a successful generic Python example cannot upgrade a native prover, verifier, solver, hardware integration, or third-party adapter from **UNKNOWN**.

8.7 Release maintenance rule

For each release candidate, update this matrix only from evidence at the exact release coordinate. A new platform claim requires, at minimum, a named component and version, the native or interpreted execution boundary, the OS and machine family, the checking mechanism, positive and adversarial results, and the exact hosted run or reproducible local receipt. When any of those dependencies changes, prior evidence remains historical and current support returns to **UNKNOWN** until it is re-established.